

長庚大學資訊管理學系

碩士論文

Department of Information Management

Chang Gung University

Master Thesis

運用混合差分進化與模擬退火演算法於

全通路零售共享容量路徑問題

Applying Hybrid Differential Evolution and Simulated

Annealing Algorithms to Omni-channel Retail with

Sharing Capacity Routing Problem

研究生：呂穗昇

Graduate Student： Sui-Sheng Lu

指導教授：林詩偉 博士

Advisor： Shih-Wei Lin, Ph.D.

中華民國 113 年 7 月

January 2024

摘要

隨著全通路零售理念的興起，零售行業正在經歷全面轉型。為了提供更完善的購物體驗，零售商正不斷擴展多樣的銷售通路，並努力整合這些通路。這種轉變有助於提升整體銷售額，但同時也需要謹慎考慮成本的因素。在這樣的背景下，優化物流問題變得尤為關鍵。

本研究首先提出一種混合差分進化與模擬退火演算法，來求解帶有容量限制車輛路徑問題(Capacitated Vehicle Routing Problem, CVRP)。然後，更進一步地利用這一演算法針對全通路零售與共享容量路徑問題(Shared Capacity Routing Problem, SCRP)進行求解。研究探討了在固定時段內，如何有效派遣運輸車輛於固定路線，以達到最佳運輸效果。特別關注實體倉庫在補給實體店面時，能否合理利用運輸車輛的閒置容量，以提高物流效益。通過在固定路線上巧妙利用閒置容量，以及考慮到順道經過店面的特點，實現物流成本和運送時間的節省，同時提升顧客對物流服務的滿意度。

本研究先求解帶有容量限制車輛路徑問題(Capacitated Vehicle Routing Problem, CVRP)，結果顯示，所求解之答案優於其他一些最先進的方法以及原本結合前的差分進化。再來針對全通路零售與共享容量路徑問題(Shared Capacity Routing Problem, SCRP)進行求解，所提

出之演算法求解小型題庫時，可獲得跟 Gurobi 求得的最佳解相同或更好。而針對大型題目時，所提之演算法可以得到比差分進化演算法更好的答案且使用更少的時間。

關鍵詞:差分進化演算法、元啟發式、共享容量路徑問題、具有容量車輛路徑問題、全通路零售



Abstract

With the rise of omni-channel retailing, the retail industry is undergoing a complete transformation. In order to provide a better shopping experience, retailers are expanding and integrating a wide range of channels. While this shift can help boost overall sales, it also requires careful consideration of the cost factor. Against this background, optimizing the logistics problem becomes particularly critical.

In this study, a hybrid differential evolution and simulated annealing algorithm is proposed to solve the Capacitated Vehicle Routing Problem (CVRP). Then, this algorithm is further utilized to solve the All-Access Retail and Shared Capacity Routing Problem (SCRCP). The study investigates how to efficiently dispatch transportation vehicles on a fixed route during a fixed period of time in order to achieve the best transportation results. Particular attention is paid to whether the idle capacity of transportation vehicles can be reasonably utilized when physical warehouses are replenishing physical stores to improve the logistics efficiency. By cleverly utilizing the idle capacity on fixed routes and taking into account the characteristics of passing by stores on the way, we can achieve savings in logistics costs and delivery time, and at the same time enhance customer satisfaction with logistics services.

This study first addresses the Capacitated Vehicle Routing Problem (CVRP). The results indicate that the solutions obtained are superior to those of some state-of-the-art methods as well as the original differential evolution algorithm before integration. Next, the study focuses on solving

the Shared Capacity Routing Problem (SCRП). The proposed algorithm, when solving small datasets, achieves solutions that are as good as or better than those obtained by Gurobi. For large-scale problems, the proposed algorithm provides better solutions than the differential evolution algorithm and requires less time.

Keywords: Differential Evolution, Metaheuristics, Shared Capacity Routing, Capacitated Vehicle Routing, Omnichannel Retail.



目 錄

摘要.....	I
Abstract.....	III
目 錄.....	V
圖目錄.....	VIII
表目錄.....	X
第一章 緒論.....	1
1.1 研究動機.....	1
1.2 研究目的.....	3
1.3 章節結構.....	4
1.4 研究流程.....	5
第二章 文獻探討	7
2.1 全通路零售與共享容量路徑問題結合	7
2.2 全通路零售與共享容量路徑問題與帶有容量限制車輛路徑問題之相互關係.....	9
2.3 帶有容量限制車輛路徑問題求解方法	10
2.4 差分進化演算法.....	11
2.5 離散型 DE	13

2.6 混合技術 DE	14
2.7 模擬退火法	15
2.8 混合算法 SA	16
第三章 研究方法	18
3.1 定義問題範圍	18
3.1.1 CVRP 問題範圍定義	18
3.1.2 全通路共享路徑問題範圍定義	19
3.1.3 SCRP 模型	21
3.2 演算法設計	23
3.2.1 解答編碼	23
3.2.2 擾動操作	24
3.2.3 組合差分進化與局部搜索混合模擬退火演算法	26
3.2.4 初始解生成方式	29
3.2.5 突變操作	30
3.2.6 交配操作	32
3.2.7 局部搜索	33
3.2.8 選擇操作	39
第四章 實驗測試與結果分析	42
4.1 CVRP 題庫	42

4.2 參數設定.....	43
4.3 參數實驗.....	43
4.4 CVRP 測試結果	45
4.5 SCRP 測試結果.....	49
4.5.1 生成 SCRP 題庫.....	50
4.5.2 驗證數學模型與演算法效益分析	51
4.5.3 分析實驗結果	53
第五章 結論與未來展望	61
5.1 研究結果與貢獻.....	61
5.2 未來相關探討議題.....	63
參考文獻.....	64

圖目錄

圖1 SCRP 範例.....	3
圖2 研究流程圖	6
圖3 解答編碼.....	24
圖4 The pseudocode of Perturbation Move.....	25
圖5 擾動範例.....	26
圖6 The pseudocode of the CDELSSA algorithm.....	28
圖7 初始解生成方式範例	30
圖8 The pseudocode of Mutation	31
圖9 突變範例.....	31
圖10 The pseudocode of Crossover.....	32
圖11 交配操作範例.....	33
圖12 The pseudocode of Local Search.....	36
圖13 The pseudocode of Strong Drop One Point.....	36
圖14 The pseudocode of Drop One Point Infeasible.....	37
圖15 The example of Store_Turnto_Transfer.....	37
圖16 The example of Swapping Algorithm.....	38
圖17 The example of Strong Drop One Point	38

圖 18 The example of Drop One Point Infeasible	39
圖 19 The example of Store_Turnto_NoTransfer.....	39
圖 20 選擇操作虛擬碼	41
圖 21 SCRP A-n80-k10 RouteCap8 MaxTransfer4適應值圖	57



表目錄

表1 直交數組和獲得的平均相對百分比偏差	44
表2 不同水準參數所得到的平均相對百分比偏差	44
表3 A 題庫結果.....	45
表4 B 題庫結果.....	46
表5 P 題庫結果	47
表6 E、F、M 題庫結果	48
表7 CDELS 與 CDELSSA 面對 CVRP 輸贏次數與平均時間結果	49
表8 CDELS 與 CDELSSA 面對 CVRP 適應值 ARPD t 檢定	49
表9 SCRP 小題庫結果.....	51
表10 SCRP 小題庫平均花費秒數.....	52
表11 SCRP 大題庫結果.....	53
表12 SCRP 大題庫平均花費秒數.....	53
表13 CDELS 與 CDELSSA 面對 SCRP 題庫輸贏次數結果.....	54
表14 CDELS 與 CDELSSA 面對 SCRP 題庫適應值 ARPD t 檢定	55
表15 CDELSSA 平均節省分析.....	55
表16 SCRP 加入時間終止條件小題庫結果	57
表17 SCRP 加入時間終止條件小題庫平均花費秒數	58

表18 SCRP 加入時間終止條件大題庫結果	58
表19 SCRP 加入時間終止條件大題庫平均花費秒數	59
表20加入時間終止條件面對 SCRP 題庫總輸贏次數結果	59
表21 加入時間終止條件面對 SCRP 適應值 ARPD t 檢定.....	60
表22 CDELSSA 加入時間終止條件平均節省分析	60



第一章 緒論

網路科技的急速發展促使線上購物和社交媒體近年來蓬勃發展，從而整體零售業的零售模式產生大幅改變。最初零售商僅限單一通路，但隨著網路興起多種通路被逐步添加至現有通路形成多通路零售，而現今最前沿的零售模式已演進為全通路零售，這建立於多通路零售的基石之上。

為了將求解全通路零售共享容量車輛路徑問題，本章先在1.1節介紹研究動機，再於1.2節說明研究目的，然後在1.3呈現章節架構，最後在1.4顯示研究之流程。

1.1 研究動機

Rigby [1]將全通路零售描述為一種將實體店面的優勢與網上購物的豐富信息體驗相融合的綜合銷售體驗。Kamel, et al. [2]進一步補充指出，真正的全通路體驗涵蓋了以客戶希望的方式、時間和地點提供服務，包括商品的購買和退貨。簡而言之，全通路概念意味著整合各種銷售通路，以便以客戶選擇的方式、定制的方式提供服務。

全通路零售之目的是以提升顧客購物體驗作為改進的核心方向，讓消費者能輕鬆自主無縫的切換不同通路，以多樣自由的方式提升消

費時的感受[3]，同時無縫的購物旅程可以加深對於零售業的支持度以保持長久關係[4]。

與多通路零售不同，真正的全通路體驗意味著一筆交易可以跨足多個通路。例如，客戶可以在線上購買產品，然後在供應商的實體零售店取貨。因此，實施真正的全通路策略需要完全整合離線和線上購物體驗[1, 5]。當前商業模式正在由傳統的多通路零售轉變為更為全面的全通路零售[6]。為了激發消費者的購物熱情，過去十年中，眾多零售業已逐漸實現不同通路的緊密結合[7]。

運輸涵蓋了產品從製造到最終物流配送的所有階段，這些階段的成本通常占總成本的10%至20%。透過電腦計算來規劃配送過程，一般可以在全球運輸成本中節省5%至20%[8]。在零售轉型的背景下，如何有效地規劃物流配送路線以降低成本成為一個重要的研究議題，值得深入探討。Paul, et al. [9]提出針對在全通路零售下建立共享容量路徑的新問題，全通路共享路徑問題(Shared Capacity Routing Problem, SCRP)，並使用元啟發式演算法求解此問題，這正是本研究所致力於解決的核心問題。圖1為 SCRP 的一個範例，該範例擁有五間實體商店，專門時段為實體商店補貨的路線稱為固定路線，而當有線上顧客有在實體商點領貨需求時，進行配送的路線稱為彈性路線，在沒有共享容量時，兩條路線總需要三台車進行配送，並且會重複拜訪商店3

與商店4，但在有共享容量的條件下，剛好固定路線剩餘容量5可以幫忙載送彈性路線中商店4的需求，於是將車輛數節省為兩台即可配送，不僅可以有效降低成本還能避免重複拜訪。

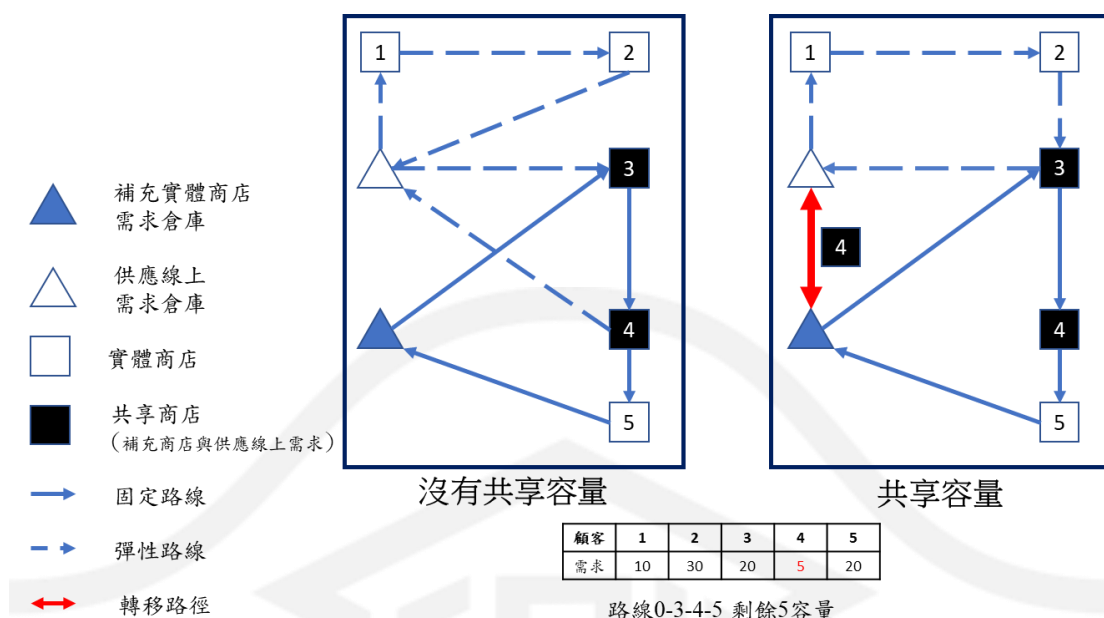


圖1 SCRP 範例

1.2 研究目的

本研究首先提出了一種優於最新 CVRP 方法的解決方案，再更進一步應用於全通路零售，以解決 SCRP 問題。研究的核心目標是在減少成本的基礎上，研究如何有效實現不同通路倉庫之間的貨物共享。

本研究之主要目的包含：

- (1) 相關文獻收集整理
- (2) 提出優於2023年最佳 CVRP 方法的改良演算法

(3) 針對 SCRP 問題提出有效率求解問題的演算法

(4) 分析演算法之效能

1.3 章節結構

第二章為文獻回顧，依循研究動機搜尋彙整相關文獻。首先在2.1節中先找出全通路零售與 SCRP(Share Capacity Routing Problem)問題之研究價值，以了解當前之研究方向，並確認零售業在施行全通路零售後所會享受其帶來的益處，隨後延續益處帶來的大量商品需求，並在此節中提及 SCRP 主要目標。再來2.2節中將與介紹 SCRP 與 CVRP 關係，並且分析近年學者對於 CVRP 的研究焦點。最後於2.3節中整理分享學術界對於 CVRP 問題的求解方法。

第三章專注於研究方法，首先描述問題，然後提出相應的解決演算法。在3.1節界定 CVRP 和 SCRP 的問題範圍。基於這些定義，3.1.3節將呈現 SCRP 的混合整數線性規劃(Mixed-Integer Linear Programming, MILP)模型，3.2節則著重於演算法設計，其中3.2.1節解釋了解決方案的編碼方法，3.2.3節詳細介紹了組合差分進化與局部搜索混合模擬退火演算法(Combinatorial Differential Evolution with Local Search hybridize Simulated Annealing Algorithm, CDELSSA)的完整流程，接下來，3.2.4節闡述初始解生成方法，而3.2.5節至3.2.8節分別討

論了突變、擾動、交配、局部搜索及選擇過程。第四章則將比較新提出的演算法處理 CVRP 的結果，再使用 GUROBI[10]求解器對 SCRP 的小型題庫進行求解，以驗證演算法的正確性。

第四章聚焦於實驗測試與結果分析，將提出的演算法和基準演算法進行比較，此過程將先在4.1節中說明 CVRP 題庫，4.2節進行參數設定。4.3節參數校正，4.4節將進行 CVRP 測試，4.5節對提出演算法進行 SCRP 測試並分析實驗結果，4.5.1節生成題庫，4.5.2節驗證數學模型並分析演算法的效益，最後在4.5.3節分析實驗結果。

第五章為結論與未來展望，針對本研究實驗結果進行總結並提出在未來可以研究的方向，在5.1節將討論實驗結果是否符合預期，探討結果背後的原因，並強調研究的貢獻。然後在5.2節根據前一節見解，更進一步探討未來可能的研究議題，並提出對本研究的潛在改進方向。

1.4 研究流程

本研究之研究流程如圖2所示。依循研究動機查詢相關文獻，接著從相關文獻去設想研究目的與研究架構以及對問題範圍進行定義，再針對問題對演算法展開設計，還有撰寫程式與例題測試，並且分析與解釋實驗結果，最後再對本研究做最終結論和未來可以改進或更進

一步研究的建議。

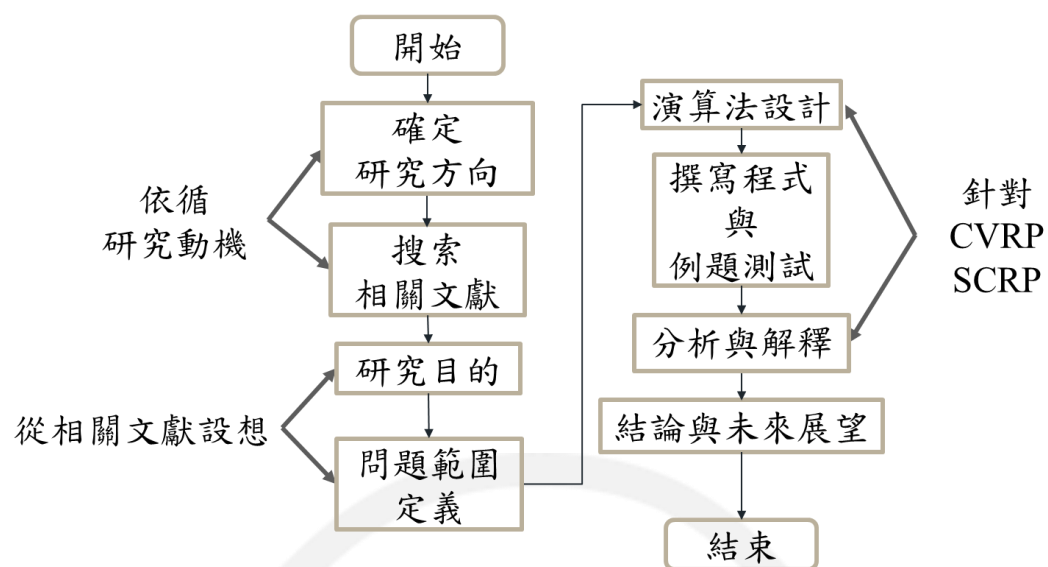


圖 2 研究流程圖

第二章 文獻探討

通路之定義不局限於店面凡是可以與顧客接觸之管道皆屬之，如行動通路是通過行動裝置進行購物[11]。

本章先在2.1節說明目前全通路零售研究現況，並且說明零售模式改變帶來的效益，接著查看尚需要研究之範圍領域，最後因為此議題的重要性被看重，所以將支持本研究的價值。在2.2節中介紹 SCRP 與 CVRP 相互關係，並指出目前學者主要研究方向。本研究會先針對 CVRP 問題提出優於目前最新穎的演算法，再對 SCRP 進行求解。最後在2.3節將特別關注於帶有容量限制車輛路徑問題的求解方法。

2.1 全通路零售與共享容量路徑問題結合

2018年全通路管理在學術研究引起更多關注，呈現強勁的發展趨勢。根據 Web of Science，其中前三個研究領域分別是商業經濟學、運營研究管理科學和計算機科學。在發表全通路管理相關論文的前10個期刊中，包括零售、營銷、管理科學等領域的期刊[12]。在研究的各個面向中，大部分都致力於探討如何透過提升全通路零售，為零售業帶來更多的好處。

將商業模式由多通路零售轉變為全通路零售已成趨勢。提供線上

購物並到實體商店提貨的服務不僅有助於提升實體商店的業績和客流量，更能促使顧客在取貨過程中產生額外消費[12]。此外，網路上的顧客瀏覽商品時產生的興趣容易轉移到實體商店，進而增進實際購買行為[4]。相較於傳統零售觀念，一般認為基於一時興起的購物會增加收入，然而實際情況卻與此相反。當消費者有意且主動地搜索商品時，這反而促使他們更傾向購買其他物品[13]。

近年來，線下零售商增加線上通路對零售商的整體業績有正面影響[14-16]。線上通路的吸引力可能來自於開拓從未使用過離線通道的新客戶群[14]，或者是由於更大的便利性（例如，搜索或旅行成本的降低）導致的在線資本的增長[15]，或者業務轉移自競爭對手的超市和其他替代品，如餐廳[16]，進而產生新的業務。

Kumar, et al. [17]研究了一家在線時尚服裝零售商增設實體店鋪的影響。結果顯示，實體店的開設提升現有在線客戶的線上購物，尤其當客戶離實體店愈遠時，效果更為明顯。作者強調實體店的參與效應，即實體店引起的互動機會，是促使在線購物增加的主要原因。同時，允許顧客在線購物並在實體店退貨對全通路零售商的線上銷售產生重要影響，並且開設實體店鋪有助於提高整體銷售額。

然而，在 Cai, et al. [12]研究發現，現有文獻中對新技術在全通路管理中的作用缺乏深入研究。隨著數字化、人工智慧、虛擬現實、移

動應用和支付技術的進步，創造了全通路零售的機遇。這些技術使得線上和線下購物之間的界線變得模糊，改變了零售格局。認為未來的研究有很大的發展空間，特別是在供應鏈上共享概念方面，例如在數據、成本和庫存共享方面的合作競爭關係[18]。在此背景下，Paul, et al. [9]針對成本與資源共享提出了全通路共享路徑問題(Shared Capacity Routing Problem, SCRP)，主要目標是建立一個靈活的時間表，同時考慮轉運，以最大程度地利用固定時間表中的空間容量，以達到總成本的最小化，這正是本研究所致力於解決的核心問題，詳細問題定義會在3.1.2節詳細說明。

2.2 全通路零售與共享容量路徑問題與帶有容量限制

車輛路徑問題之相互關係

全通路零售與共享容量路徑問題(Shared Capacity Routing Problem, SCRP)為兩倉庫獨立路線車輛容量間的共享，當在沒有任何共享時，所求解問題即為帶有容量限制車輛路徑問題(Capacitated Vehicle Routing Problem, CVRP)問題[9]，所以在本研究會先對 CVRP 問題進行求解再求解 SCRP 問題。

VRP 也稱之為 CVRP[19]。在這個問題中會考慮容量的限制，服

務顧客的總需求不得超過車輛的容量，並且只考慮單一商品[20]。更完整的定義於3.1.1節介紹。

學者一直對解決 CVRP 變體表現出濃厚的興趣。每年約有26篇相關論文發表，為因應實際挑戰，研究者們通常會研究多個問題變體。其中，CVRP 被視為最簡單的變體之一，因此成為整合其他變體研究的理想起點[21]。

2.3 帶有容量限制車輛路徑問題求解方法

在面對帶有容量限制車輛路徑問題時，使用各式的元啟發式演算法成為主流[22]，如：模擬退火法(Simulated annealing algorithm, SA)[23]、禁忌搜尋法(Tabu search, TS)[24]、基因演算法(Genetic Algorithm, GA)[25]和差分進化演算法(Differential Evolution, DE)[26]。這三種都有其各自的特色，SA 使用有機率接受較差的解的機制，TS 利用清單來記憶近期解，避免重複拜訪以逃離局部解，GA 應用選擇交配突變的方式運行，DE 則是與 GA 相似但其運行順序相反。然而近年許多學者以混合兩種元啟發式演算法進行研究[27]，如：SA 和 TS[28]、粒子群優化演算法(Particle Swarm Optimization, PSO)和 GA[29]、頭腦風暴優化演算法(Brain Storm Optimization, BSO)和 SA[30]、a hybrid Genetic Variable Neighborhood Search Algorithm

(HGA-VNS)[31]、a two-phase heuristic combining LNS with Set Partitioning (2PH)[32]、a matheuristic GRASP+VNS (MGV-CP)[33]、Differential Evolution with Local Search (DELS)[34]、Combinatorial Differential Evolution with Local Search(CDELS)[19]等。

接下來在2.4節和2.7節中會著重於介紹本研究所使用到的 DE 與 SA。

2.4 差分進化演算法

由於 CVRP 的 NP hard 性質[35]，透過精確的演算法求解 CVRP 非常耗時，尤其是當客戶數量很大時。因此，在過去十年中，人們開發了許多啟發式方法來解決 CVRP[36]，但近年來，許多研究人員專注於使用進化演算法(Evolutionary Algorithms; EA)來解決 CVRP[37]。差分進化演算法 (Differential Evolution, DE) 屬於 EA 中的一種。EA 是一種基於種群的算法種類，持有多個個體形成種群，其中解決問題的解由個體表示。模擬1859年達爾文著作物種原始中的天擇演化觀點，當中圍繞著交配遺傳、突變、和選擇，也是 DE 的核心原理，在優化問題時，會使用核心操作方法，交配與突變的方式，以進化個體並找到更好的解決方案[38];依據當前擁有解先產生多個解決方案，再尋找其他更好的結果，最後依據結果進行判斷並且沿用好的解決方

法。交配和突變目的皆是在搜尋全域解，而選擇則是讓好的解決方案留下。基於 EA 的演算法有很多種，如：GA[39]、差分進化演算法 (Differential Evolution, DE)[26]和機率分佈估計演算法(Estimation of Distribution Algorithm, EDA)[40]等。

EA 的主要優點是理想情況下不對潛在的目標做出任何先驗假設，可以處理不可微、非線性、非凸性等，甚至更複雜的數學問題，所以使用時機與範圍相當廣泛，並且進化演算法與其他較為傳統的最佳化方法相比更加穩定[41]，不過進化演算法有一個缺點，收斂效率差[42]，所以近年多數學者的研究方向皆在提升收斂效率，同時保持全局搜索能力[43]。

DE 最初是一份技術報告由 Storn [44]首次提出。在1996年5月的第一屆國際演化優化大賽中展現出其性能，該大賽與同年 IEEE 國際進化計算大會(CEC)同期舉辦[45]。自那時以來，DE 已經成為研究人員在解決不同領域的優化問題時的熱門選擇。根據 Google Scholar 的引用統計，截至2020年為止，至1996年以來 DE 的總引用次數達到20366，其中有7種變體的引用次數超過500。此外，自2005年以來，DE 的多個變體在連續幾年的 CEC 競賽中屢次進入前三名，僅在2013年未能進入前三名[46]。

差分進化算法相對遺傳算法的收斂效果更加顯著，因為差分進化

算法變異向量是由上一代差分向量生成，並與父代個體向量交叉生成新個體向量，直接與其父代個體進行選擇，而遺傳算法是根據適應度值來控制父代雜交，變異後產生的子代被選擇的概率值，在最大化問題中適應值大的個體被選擇的概率相應也會大一些[47]。相較其他進化算法，DE 擁有多個優點，例如實現容易且能生成出更優秀的結果以及空間複雜度低[47]。

2.5 離散型 DE

近年也有將 DE 應用在離散型的問題，在編碼方法上就先進行離散處理，讓演算法能直接使用離散數值進行求解，也被稱為(Discrete Differential Evolution, DDE)[48]。編碼型式可以分為兩種，二進制和整數值[49]。

近期有學者提出使用 DE 成為解決 CVRP 新穎方法，本研究將會基於此方法進行優化，此文獻是組合差分進化結合局部搜索的演算法(Combinatorial Differential Evolution with Local Search, CDELS)，首先利用父代的差分向量生成突變向量，再與父代個體向量交配生成新後代個體向量，最後再直接與父代個體進行選擇，用此方式加速演算法的聚合[19]。Souza 在未來展望部分指出可以將 CDELS 應用於其他 VRP 變體，並且增強 CDELS。於是本實驗要接續加強此演算法以提

升其性能，最終在運用到 SCRP 問題中[19]。

2.6 混合技術 DE

對 DE 進行混合其他元啟發式演算法是為了提高其性能和加速收斂速度的重要修改之一[46]。許多文獻都探討了 DE 混合化的方法。Liu, et al. [50]提出一種結合 DE 與粒子群(Particle Swarm Optimization, PSO)的混合技術，用於處理約束優化問題。Stanarevic [51]混合了 DE 和人工蜜蜂群(Artificial Bee Colony Optimization, ABC)。同樣，Sharma, et al. [52]進行 DE 和 ABC 的混合。Ustun, et al. [53]將 ABC 與 DE 進行混合，並稱之為混合優化方法(hybrid optimization method, HOM)。Worasuchep [54]使用基於對立的混合 ABC 和 DE，稱為對立式混合 ABC 和 DE 算法(opposition-based hybrid Artificial Bee Colony (ABC) with Differential Evolution (DE) algorithm, OABCDE)，克服 ABC 和 DE 單獨使用時的一些限制，提高全局搜索和收斂性能，用於解決連續優化問題。Sethanan, et al. [55]將三種局部搜索元啟發式方法(shift、exchange(SWAP)和 k-variable move)應用於 DE 中，並將其用於解決分配問題。Li, et al. [56]將 DE 與一種稱為簡化二次逼近(simplified quadratic approximation, SQA)的鄰域算法進行混合，用於約束優化問題。

2.7 模擬退火法

模擬退火法(Simulated Annealing Algorithm, SA)概念最早是源於 Metropolis 等人首次提出[57]，是一種受金屬冶煉中退火過程啟發的概率搜索方法。該技術涉及對金屬進行加熱，然後非常緩慢地冷卻，以獲得適當的晶體結構。在全局優化的背景下，這種基於模擬的技術通過物理系統與優化問題之間的類比得以說明：材料狀態對應於問題解決方案，能量對應於目標函數，基態對應於全局最佳解，而溫度對應於控制參數。在 SA 的每次迭代中，都會隨機生成一個新的解決方案。新解決方案與當前解決方案之間的差距基於與溫度成比例的概率分佈。該算法接受降低目標的所有新解決方案，但也以某一概率接受提高目標的解決方案，因此有助於逃避局部陷阱。

當目標函數評估需要大量記憶空間，例如由於涉及大量記憶的複雜模擬過程，牽涉到大維度狀態空間時，基於群體的算法可能不適用，而模擬退火是解決這類問題的正确答案。特別是當目標函數的評估非常昂貴，需要大量計算時。模擬退火的特性，如接受差解的機率隨著時間的推移而降低，使得算法在搜索過程中可以跳脫局部最優解，增加全局搜索的機會。這有助於在大量可能的解中尋找更好的解[58]。

Kirkpatrick, et al. [59]將此思維導入組合優化領域中，此後 SA 成

為一種流行的元啟發式方法來解決，隨後 Osman 將 SA 精神應用在 CVRP 問題上[60]後，逐年都有學者致力於研究提升 SA 用來解決此問題[61-63]。

2.8 混合算法 SA

特別是在過去幾年中，有研究傾向於採用混合技術來提高最佳化過程的整體性能，開發了結合元啟發式的混合演算法[64, 65]，Xiao, et al. [62]提出了變鄰域模擬退火(Variable Neighborhood Simulated Annealing, VNSA)演算法，它是變鄰域搜尋與模擬退火相結合的變體。他們在39個知名基準實例上對其進行了測試。還有用於求解 VRP 提出的 SA 混合粒子群演算法[66]。

因為 SA 存在一些問題:(1)初始解很大程度上影響了它的性能。當起點的品質不好時，結果可能會不盡人意[67]。(2)使用單點搜尋的方式來完全探索搜尋空間可能不合適[68]。(3)尋找合理解需要較長的計算時間。當問題具有高維度和許多局部最優時，這些缺點就很明顯[68]。所以 Ilhan [67]提出了一種基於群體的模擬退火(Population Based SA)演算法來求解 CVRP。它包括三個局部搜尋運算元，即交換、插入和恢復。這些運算元用於探索當前解決方案的鄰居。在隔年 İlhan [69]提出改良交配運算元的改進模擬退火演算法(simulated annealing

algorithm with crossover operator, ISA-CO)。使用一種基於群體的模擬退火演算法，並結合了四種局部搜尋運算元(交換、打亂、插入和反轉)來改善解的品質和收斂速度求解 CVRP。與基於種群的模擬退火不同，使用交叉運算元和解之間的信息交換，有助於獲得更高的收斂速度，且借助混合選擇程序，在迭代的前半部分使用 SA 有機會接受較差解，在迭代後半部分繼續收斂，獲得更好的解。並且使用田口方法計算 ISA-CO 參數的最佳值。

第三章 研究方法

本章節將在3.1節中說明並定義本研究所要求解的問題，以此更好的了解此問題所屬的範圍，在3.1.1節定義 CVRP 問題範圍，然後3.1.2節定義 SCRP 問題範圍，再接著於3.1.3節針對問題定義呈現其數學模型，再於3.2節對演算法進行設計以符合問題，並且會在3.2.1中表示解決方案的編碼方式，並於3.2.2介紹擾動操作，然後於3.2.3介紹演算法，最後在3.2.4至3.2.8中說明 CDELSSA 的詳細流程。

3.1 定義問題範圍

本研究會先改進 CDELS 演算法應用於 CVRP，緊接著再使用此改進算法去進行求解 SCRP。在3.1.1節進行 CVRP 問題範圍定義，然後在3.1.2節定義全通路共享路徑問題範圍。

3.1.1 CVRP 問題範圍定義

帶有容量限制車輛路徑問題(CVRP)可以使用完整的無向圖來定義 $G = (V, E)$ ， $V = \{c_0, c_1, \dots, c_N\}$ 是頂點集並且 $E = \{(c_i, c_j): c_i, c_j \in V, i < j\}$ 是邊集。頂點 $c_0, c_1, \dots, c_N$ 代表顧客和每個顧客 c_i 皆與非負需求 q_i 相關聯。頂點 c_0 代表著一個同質車輛容量為 Q 的車隊將要啟程的場

站。場站沒有相對應的需求。每條邊 $(c_i, c_j) \in E$ 與非負的行駛成本或距離 d_{ij} 相關聯。CVRP 旨在最小化一組車輛路線，滿足以下條件：

- (1) 每條路線均始於並結束於中央倉庫
- (2) 每位客戶被訪問一次
- (3) 路線的總需求不超過 Q
- (4) 路線的數量不超過車輛的數量 K

3.1.2 全通路共享路徑問題範圍定義

本研究問題定義為全通路零售共享容量車輛路徑問題，此問題基於 Paul, et al. [9]中的問題，在全通路零售下建立共享容量路徑問題，並使用演算法求解出此問題。首先共享策略僅針對同一家零售商實施，且其擁有兩個獨立倉庫位，不但落座於不同座標位置，還分別替線上商店與線下實體商店提供服務，再來共享對象皆為實體商店，最後共享容量為從兩種不同倉庫發車的貨車。由於擁有兩種倉庫，所以會有兩種路線，且皆為運送至實體商店，分別為：

- 固定路線

定期性供給日常線下實體商店補貨的車輛路線

- 彈性路線

當有線上顧客需求時，才配送貨品給實體商店的車輛路線

兩種路線皆會將貨物送至實體商店，所以會發生同一間商店被服務兩次的問題，於是參照 Paul, et al. [9]的方法，以解決產生的問題，加上又能同時滿足全通路零售的通路整合策略。在考慮固定路線是定期性的，而彈性路線是不固定的，所以將轉移設置成只能從彈性路線移至固定路線，希望可以藉由固定路線的空間與相同目的地來幫忙載送，以此減少時間與成本。

在兩條路線中重複出現的被視為共享商店，且當其位於的固定路線有足夠的空間可以乘載，即轉移至固定路線，否則不進行轉移。

本問題將所有商店設定為共享商店，且不計算固定路線花費車輛成本，只會探討彈性路線及轉移車輛成本，最遠兩點之間的距離一半作為轉移運輸成本。

轉移之假設為：

- (1) 車輛足以滿足所有轉移需求
- (2) 需要轉移的需求會在服務固定路線前被轉移
- (3) 轉移的需求可以拆分成不同車輛運輸

當本研究問題沒有需要轉移之商店時，將會退化成 CVRP 問題，因為僅需要考慮用彈性路線將所有商店需求滿足。

3.1.3 SCRP 模型

介紹 SCRP 模型，並且如果移除轉移共享相關限制即為 CVRP 模型。SCRP 主要限制有五點：

- (1) 每台車的服務始末點皆為倉庫
- (2) 每台車有最大容量限制
- (3) 每台車載重不可超過最大車輛容量
- (4) 所有商店皆需要被服務，或者轉移
- (5) 所有商店只能經一台車拜訪或轉移一次

集合：

V 倉庫與實體商店集合

N 實體商店集合

K 彈性路線中車輛集合

R 固定路線中的路線集合

S 可轉移的實體商店集合

S_r 固定路線中第 r 條路線的實體商店集合， $r \in R$

參數：

q_i 節點 i 的需求量， $i \in N$

Q 彈性最大車輛容量

Q' 轉移最大車輛容量

r 固定路線中的路線

E_r 固定路線中第 r 條路線的空間空間， $r \in R$

C_{ij} 從節點 i 至節點 j 距離， $i, j \in V$

F 轉移成本

決策變數：

x_{ij} $\{0,1\}$ 車輛從節點 i 服務至節點 j 為1，否則為0，

$i, j \in V$

y_i $\{0,1\}$ 節點 i 被轉移至固定路線為1，否則為0， $i \in N$

u_i 服務至節點 i 時總需求量， $i \in N$

w 從彈性路線轉移至固定路線的整數次數

目標式：

$$\min \sum_{i \in V} \sum_{j \in V} \sum_{k \in m} C_{ij} x_{ij} + FW \quad (1)$$

限制式：

$$y_i + \sum_{j \in V} x_{ij} = 1 \quad \forall i \in N \quad (2)$$

$$y_j + \sum_{i \in V} x_{ij} = 1 \quad \forall j \in N \quad (3)$$

$$\sum_{i \in N} q_i y_i \leq Q'w \quad (4)$$

$$\sum_{i \in S_r} q_i y_i \leq E_r \quad \forall r \in R \quad (5)$$

$$u_i - u_j + Qx_{ij} \leq Q - q_j \quad \forall i \in N, j \in N \quad (6)$$

$$x_{ij} \in 0,1 \quad \forall i \in V, j \in V \quad (7)$$

$$y_i \in 0,1 \quad \forall i \in N \quad (8)$$

$$w \in Z \geq 0 \quad (9)$$

$$q_i \leq u_i \leq Q \quad \forall i \in N \quad (10)$$

目標式(1)為極小化車輛成本，限制式(2)和(3)每一間商店皆會被服務或轉移一次，還有只要被服務到的商店皆會有車從此離開，並且確保車輛起點與終點皆為倉庫。限制式(4)轉移的總需求不超過可轉移最大容量。限制式(5)轉移至固定路線需求量要符合每條固定路線的空間空間。限制式(6)確保車輛載重的正確性。限制式(7)到(10)確保決策變數，達到問題的合法性[9]。

3.2 演算法設計

本研究全通路零售共享容量車輛路徑問題使用 CDELSSA 求解，透過本章節將了解演算法的理念與求解過程。在3.2.1節展現解決方案的編碼方式，於3.2.2節說明擾動操作，並在3.2.3節介紹完整 CDELSSA 流程，最後3.2.4節介紹初始解生成方法。

3.2.1 解答編碼

解答編碼分成兩部分展示於圖3中，第一部分為彈性路線的車輛拜訪順序，所以每條路線為首的位置與最後一個位置皆為廠站，1以

後的數字皆代表商店，在此共有三條路線。第二部分為此商店有無轉移至固定路線，1、4、6商店被轉移。第一部分可以表示未轉移前的車輛路徑，如果搭配上第二部分即可以表示被轉移後的路線，因為在第一部分被轉移的商店會被忽視，在解決 CVRP 問題時，第二部分會維持0，因為不進行共享轉移。範例於圖3中顯示，固定路線 R_1 剩餘容量為10，然而，彈性路線中商店1、商店4與商店6的需求可以由固定路線幫忙運送來減少成本，所以將可轉移之彈性路線商店進行轉移。

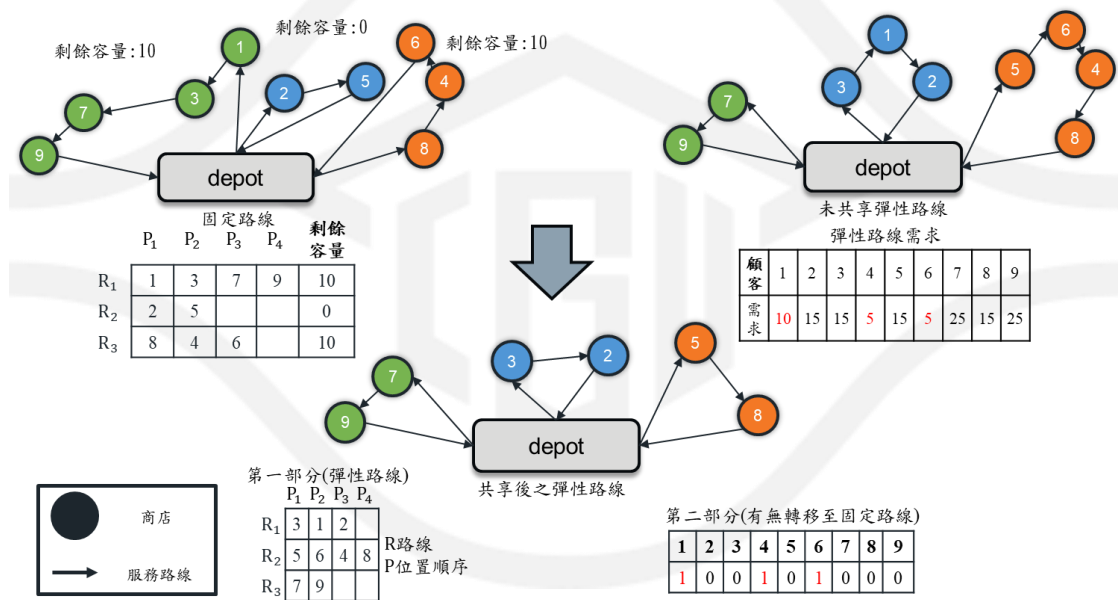


圖3 解答編碼

3.2.2 擾動操作

擾動操作(Perturbation Move Operator)的實施是基於 Souza, et al. [19]中的技術。在演算法中擾動操作起到了核心的作用，不管在突變還是交配階段皆會應用到，此擾動目的是直接考慮到人口位置資訊，

根據個體上的位置交換客戶。首先於 Z 隨機選擇一個商店 c_j ，紀錄其商店和路線位置，再來針對要擾動的個體 Y 複製個體成為 Y' ，在 Y' 中找到 c_j 和他在 Z 之所在位置，如果其所在位置於 Y' 有點位則直接進行交換，反之則進行插入動作。主要將兩個隨機商店進行交換，但也有例外是進行插入動作，詳細步驟在圖4。

Algorithm 1: The pseudocode of Perturbation Move

```

Input:  $Y, Z, (p_s, R_\ell)$ 
1   $c_j \leftarrow$  store located at  $(p_s, R_\ell)$  of  $Z$ 
2   $Y' \leftarrow Y$ 
3  if there is a store  $c_k$  located at  $(p_s, R_\ell)$  of  $Y'$  then
4       $Y' \leftarrow$  swap  $c_j$  and  $c_k$  in  $Y'$ 
5  else if the route of store  $c_j$  in  $Y'$  has more than one store
    then
6       $p_{last} \leftarrow$  last position of route  $R_\ell$  of  $Y'$ 
7       $Y' \leftarrow$  remove the store  $c_j$  and reinsert it at  $(p_{last} + 1, R_\ell)$  of  $Y'$ 
8  Return  $Y'$ 

```

圖4 The pseudocode of Perturbation Move

擾動的範例如圖5中顯示有正常情況和例外情形，正常情況下如選定 $X_{r_{2,g}}^i$ 中 $c_j = 8$ 和其對應於 $V_{i,g+1}^1$ 中的位置有商店6，直接將兩者互相交換即可，例外為當 $c_j = 12$ 時，對應於 $V_{i,g+1}^1$ 中的位置皆沒有商店存在，所以採取插入的方始進行擾動，將 c_j 插入對應位置路線中最後一位，如果在 $V_{i,g+1}^1$ 中被移動路線中後面要自動往前補，例如 $c_j = 12$ 被移動後，商店11需要往前補齊。

$$V_{i,g+1}^{m+1} \leftarrow \text{PerturbationMove}(V_{i,g+1}^m, X_{r_2,g}^i, (p_s, R_\ell))$$

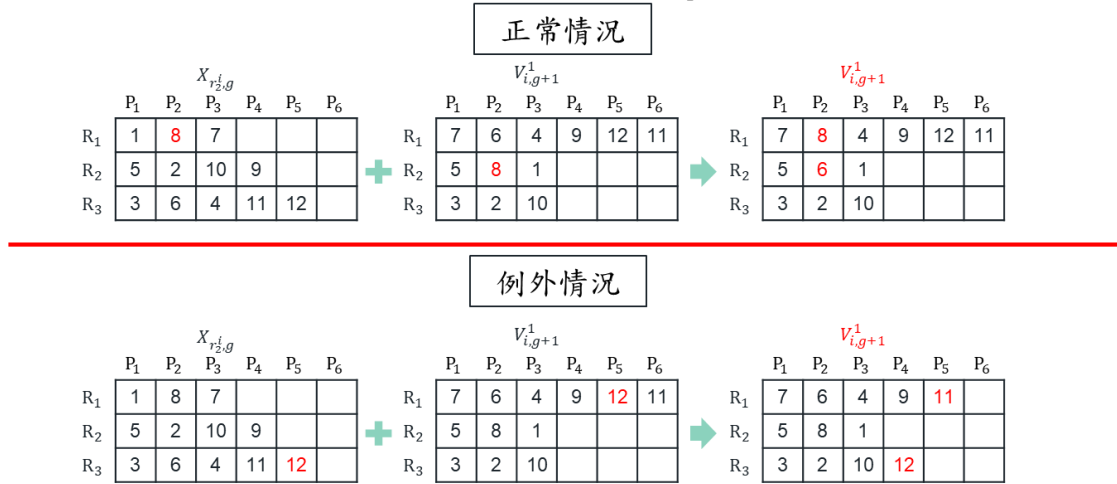


圖5 擾動範例

3.2.3 組合差分進化與局部搜索混合模擬退火演算法

組合差分進化與局部搜索混合模擬退火演算法(Combinatorial Differential Evolution with Local Search hybridize Simulated annealing algorithm, CDELSSA)基於最新穎離散 DE 演算法進行改良[19]。由四大元素組成，第一部分為差分進化演算法突變與交配操作，增加解決方案多樣性，引領演算法在全域可行解空間中前進，於3.2.5與3.2.6節有更詳細介紹。第二部分局部搜索策略，負責搜索當前鄰域空間解，並且提升解之質量與解決方案聚合速度，完善之說明於3.2.7節進行講解。第三第四部份為與 İlhan [69]相似的混合選擇程序機制，在迭代的前半部分跳脫區域最佳解之後繼續往全局最佳解收斂。第三部分在選擇(select)部分，為混合選擇程序機制前半段使用 SA 機制更新世代

解，會在選擇階段保有特定機率接受較差的解決方案，以跳脫區域最佳解，3.2.8節中會說明觸發時機。最後一部分在 select 階段後執行，為混合選擇程序機制後半段，將每次產生世代解繼續往全局最佳解收斂，於3.2.8節詳細說明。

四大元素統整起來形成 CDELSSA，確切演算法步驟於圖6呈現，首先導入無向圖 G 、車容量 Q 、最大車輛數 K 、突變參數 F 、交配參數 CR 、個體數量 NP 、懲罰值 $Penalty$ 、同溫度迭代次數(T_n)和當前溫度(T_k)，並將世代數 g 和降溫幾次沒有更新的計數器(Temperature No Update, TNU)預設為0，接著初始化人口種群，並且將上一次降溫前更新最佳解適應值(Last Temperature Best Cost, $LTBC$)賦予為 P_0 ，第一步進入迭代階段執行 T_n ，每一次會先將 P_{g+1} 賦值為0，緊接著實施 NP 次數個體迭代，第二步對每個個體展開交配和突變操作以進化種群，第三步進行 Local Search 來提高解決方案的質量，細節於3.2.7節會進行說明，第四步是 select 搭配 SA 機制將有機率接受較差解，展示於3.2.8節，從第二步到此步驟會執行 NP 次。第五步為世代解與當前最佳解比較，並且一定只接受較佳解，第六步將 g 進行加一，到此結束第一步道第六步 T_n 迭代。第七步則是進行 SA 降溫將 T_k 乘上 $Fraction$ 降溫參數，步驟八判斷此次降溫是否有優於 $LTBC$ ，有的話將 TNU 加一且更新 $LTBC$ ，反之則歸零，重複步驟一到八，直到 $TNU \leq$ 最大降

溫次數沒更新($N_{\text{non-improving}}$)，得到最終結果。終止條件為 SA 降溫幾次沒有更新。因為要與 CDELS[19]進行比較所以針對其終止條件，修改成與 CDELSSA 一致，所以將其終止條件設為最大降溫次數沒更新($N_{\text{non-improving}}$)乘同溫度迭代次數(T_n)。

Algorithm 2: The pseudocode of the CDELSSA algorithm

```

Input:  $G = (V, E)$ ,  $Q$ ,  $K$ ,  $F$ ,  $CR$ ,  $NP$ , Penalty,  $T_n$ ,  $T_k$ 
1   $g \leftarrow 0$ 
2   $TNU \leftarrow 0$ 
3   $P_0 \leftarrow \text{Generate Initial Population}(G, NP, K, Q)$ 
4   $LTBC \leftarrow P_0$ 
5  do
6      For  $t = 1$  to  $T_n$  do
7           $P_{g+1} \leftarrow \emptyset$ 
8          For  $i = 1$  to  $NP$  do
9               $Xr_{1,g}^i, Xr_{2,g}^i \leftarrow \text{randomly select } Xr_{1,g}^i, Xr_{2,g}^i \in P_g, i \neq r_1^i \neq r_2^i$ 
10              $V_{i,g+1} \leftarrow \text{Mutation}(Xr_{1,g}^i, Xr_{2,g}^i, F)$ 
11              $U_{i,g+1} \leftarrow \text{Crossover}(X_{i,g}, V_{i,g+1}, CR, K)$ 
12              $U'_{i,g+1} \leftarrow \text{Local Search}(U'_{i,g+1})$ 
13              $X_{i,g+1} \leftarrow \text{Selection}(X_{i,g}, U'_{i,g+1}, T_k, NP, P_{g+1})$ 
14             If cost of  $P_{g+1}$  is less than  $X_{best}$  and then
15                  $X_{best} \leftarrow P_{g+1}$ 
16              $g \leftarrow g + 1$ 
17              $T_k = T_k \times \text{Fraction}$  //The reduction of the temperature
18             If  $LTBC \leq \text{cost of } P_g$ 
19                  $TNU \leftarrow TNU + 1$ 
20             Else
21                  $LTBC \leftarrow \text{cost of } P_g$ 
22                  $TNU \leftarrow 0$ 
23 while  $TNU \leq N_{\text{non-improving}}$ 
24 Return  $X_{best}$ 

```

圖 6 The pseudocode of the CDELSSA algorithm

3.2.4 初始解生成方式

本研究遵循 Souza, et al. [19] 的策略進行初始化解生成。初始解需要創建 NP 個隨機個體， N 為個體數量加上場站， NP 為3倍 N 個數依據 Souza, et al. [19] 推薦，每個個體代表 CVRP 的一個解決方案。

初始解由兩個步驟組成，此兩個步驟分別為：

- (1) 每個個體先建立 K 條路線(K 為最大車輛數量)，路線由商店集合不重複隨機選取組成，並且確保每條路線不會超過車輛限制 Q 也不包含場站
- (2) 如果有任何商店無法被可行的放入路線，皆將此商店全部放入隨機一條路線中，如果有此情況發生，此個體及為不可行解需要加上懲罰值。

如果步驟二成立，這將會造成個體會變成 $K-1$ 條可行路徑和1條不可行路徑，所以目標值要添加懲罰值。如圖7中商店皆為隨機放入路線，商店9被選到放入路線 R_3 ，但因為違反重量限制，因此隨機選擇插入路線 R_2 ，放入後依舊違反，所以在該路線適應值加入懲罰值。

商店	1	2	3	4	5	6	7	8	9
需求	10	30	5	20	20	10	20	30	5

車數量:3
車容量:50

	P ₁	P ₂	P ₃	P ₄	
R ₁	3	1	2		45
R ₂	5	6	4		50
R ₃	7	8			50

	P ₁	P ₂	P ₃	P ₄	
R ₁	3	1	2		45
R ₂	5	6	4	9	55
R ₃	7	8			50

加懲罰值

圖7 初始解生成方式範例

3.2.5 突變操作

在進行突變操作(Mutation Operator)時，我們參照了 Souza, et al. [19]的指引。每一次執行首先會隨機選擇兩個個體參與突變， $V_{i,g+1}^1$ 會是第一個個體 $X_{r_1,g}^i$ 的複製體，會針對此個體隨機挑選 $\lceil \frac{N}{2} \times F \rceil$ 個點位進行擾動，其中參數 F 為影響個體間變動程度的關鍵。除以二的原因是單次位置交換可以改變個體的兩個位置，因此 $F \times size/2$ 次交換足以完成改變個體。由於每個 CVRP 解決方案都包含每間商店，因此我們可以從清單中 L 所有商店中 $V - \{c_0\}$ 選擇 c_j 。隨機在 L 擇一個商店 c_j ，隨即該商店會被刪除自 L 並蒐索在 $X_{r_2,g}^i$ 中路線位置。由於路線可以有不同的大小，因此所選的商店位置在 $X_{r_2,g}^i$ 可能會不存在於 $V_{i,g+1}$ 。在這種情況下，商店 c_j 會被附加在 $V_{i,g+1}$ 相應路線的最後一個位置，其中 Souza, et al. [19]為了維持當前車輛數量，僅在操作前 c_j 路線車輛數量大於1時才執行此操作，但本研究希望可以取得更少車輛，於是將此限制移除，並避免車輛超過使用上限。參數 F 會依照 Souza, et al. [19]

建議數值0.7執行，詳細步驟在圖8。範例於圖9先將隨機選中的個體一複製成突變個體 V ，再將選中商店8與突變個體及個體二進行擾動，圖9中記錄商店8在突變個體 V 中位置，並與個體二與商店8進行交換，及可以得到最終突變個體。。

Algorithm 3: The pseudocode of Mutation

Input: $X_{r_1,g}^i, X_{r_2,g}^i, N, F$.

- 1 $V_{i,g+1}^1 \leftarrow X_{r_1,g}^i$
- 2 $L \leftarrow c_1, \dots, c_N$
- 3 **For** $m \leftarrow 1$ to $\lceil \frac{N}{2} \times F \rceil$ **do**
- 4 $c_j \leftarrow$ a random store of L
- 5 $L \leftarrow L - \{c_j\}$
- 6 $(p_s, R_\ell) \leftarrow$ store c_j position p_s in route R_ℓ of $X_{r_2,g}^i$
- 7 $V_{i,g+1}^{m+1} \leftarrow \text{PerturbationMove}(V_{i,g+1}^m, X_{r_2,g}^i, (p_s, R_\ell))$
- 8 **return** $V_{i,g+1}^m$

圖8 The pseudocode of Mutation

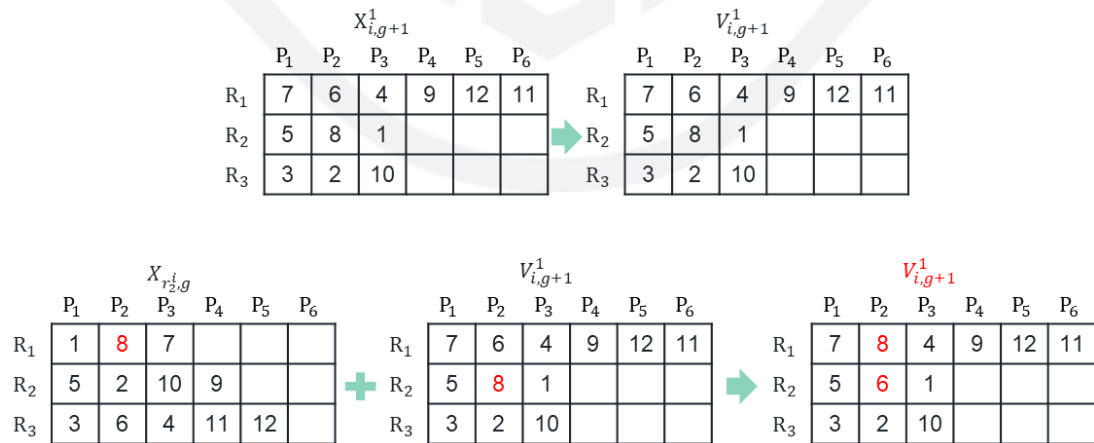


圖9 突變範例

3.2.6 交配操作

本研究的交配操作(Crossover Operator)過程採用了 Souza, et al. [19]中的方法。如圖11為了避免交配後和原先目標個體的解相同，會先執行初始化，將上一世代個體 $X_{i,g}$ 複製成 $U_{i,g+1}^0$ ，再將其與突變個體 $V_{i,g+1}$ 進行一次擾動轉變成 $U_{i,g+1}^1$ ，再針對此個體中每個路線中商店進行均勻分佈的隨機數 $[0.0,1.0]$ 判斷，如果小於等於參數 CR 且隨機選擇商店 c_q 沒有在第一次擾動執行過，則進行擾動操作，反之則終止迴圈並回傳當前個體結果。上述動作會對個體中的所有路線重複執行。詳細步驟在圖10。參數 $CR \in [0,1]$ 會影響到交配的變動程度，本研究會依照文獻建議數值0.7執行[19]。

Algorithm 4: The pseudocode of Crossover

```

Input:  $X_{i,g}$ ,  $V_{i,g+1}$ ,  $CR$ ,  $K$ .
1  Initialize  $U_{i,g+1}^0$  as a clone of  $X_{i,g}$ 
2   $c_j \leftarrow$  random store of  $V_{i,g+1}$ 
3   $(p_s, R_\ell) \leftarrow$  position  $p_s$  associated to store  $c_j$  on route  $R_\ell$  of  $V_{i,g+1}$ 
4   $U_{i,g+1}^1 \leftarrow$  PerturbationMove( $U_{i,g+1}^0$ ,  $V_{i,g+1}$ ,  $(p_s, R_\ell)$ )
5   $m \leftarrow 1$ 
6  for  $\ell \leftarrow 1$  to  $K$  do
7      forall  $c_q$  in position  $p_s$  on route  $R_\ell$  of  $V_{i,g+1}$  do
8          If  $rand() \leq CR$  and  $c_q$  was not swapped then
9               $U_{i,g+1}^{m+1} \leftarrow$  PerturbationMove( $U_{i,g+1}^m$ ,  $V_{i,g+1}$ ,  $(p_s, R_\ell)$ )
10              $m \leftarrow m + 1$ 
11  Return  $U_{i,g+1}^m$ 

```

圖10 The pseudocode of Crossover

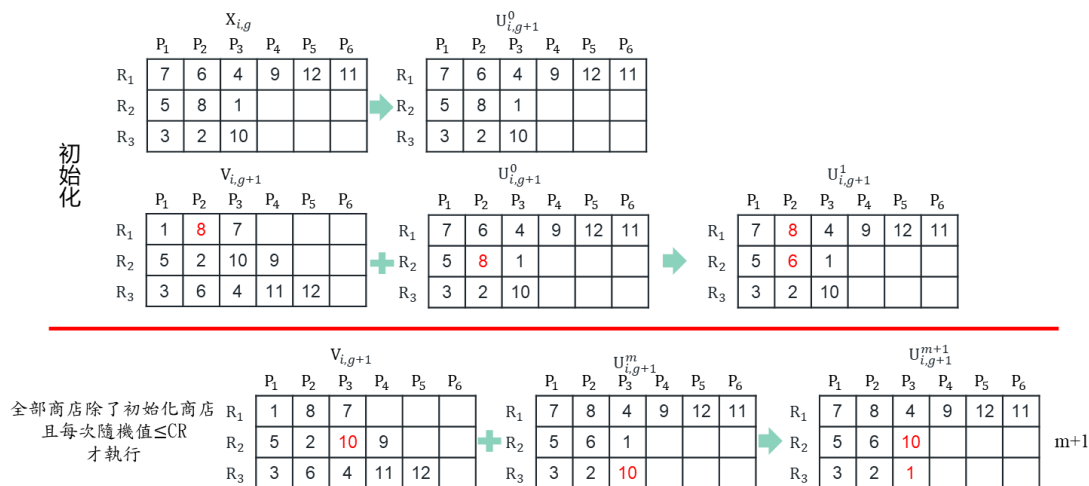


圖11 交配操作範例

3.2.7 局部搜索

局部搜索(Local Search)由五部分組成，目的在於進一步改善已獲得的解並探索鄰近的更佳解決方案，局部搜索用於突變和交配過後，根據沒有改進計數器 $it_no_improvement$ 有不同的動作。詳細步驟在圖12。

(1) 商店轉移共享階段(Store_Turnto_Transfer)：隨機選取商店進行共享，

如果違反重量限制式亦或是重複選取，則不執行。範例於圖15，

會隨機選取商店 c_q ，此處選到商店1，會先判斷是否已是轉移商店，

且轉移是否會違反重量限制式，如皆不是則會將此商店轉移。

(2) 交換策略階段(Swapping Algorithm)：在這一階段，我們專注於解空

間中商店進行策略性交換。我們詳細評估每對點位之間的交換效

益。關鍵在於，只要不觸犯既定的約束條件，最終都將一定執行

並優先選擇能產生最大效益的交換操作，最差情況是維持不變。

如圖16中交換商店2和商店5將有助於降低適應值。

- (3) 點位強移階段(Strong Drop One Point)：在這一步驟中，首先選取一位商店，並從其當前所在的路線中移除。接著，嘗試將該商店重新分配到其他的路線中，並對每一條可能的路線的每一個位置進行試驗，目的是尋找一個可以使得整體成本最低的位置，此操作僅在個體成本降低的情況下執行。如果沒有找到合適的位置，則該商店的位置保持不變，一旦這位商店的操作完成，就換到下一位商店執行相同的過程，直到所有商店都進行過此操作。在整個過程中，都嚴格遵循所有的約束條件。在 Souza, et al. [19]中為了保持個體目前的車輛數，如果一條路線只有一個客戶，則保留該客戶，但本研究將其限制移除希望可以取得更少車輛，並避免車輛超過使用上限。詳細步驟在圖13。如圖17將商店7從路線 R_1 移置路線 R_3 ，可以有效降低適應值。

- (4) 不可行解優化階段(Drop One Point Infeasible)：此階段的策略目的是對那些當前不可行的解進行精確優化。從整體個體中，對隨機選定的非合法路線 R_r 進行深度探究。如果確定存在某商店 c_j 可以成功地轉移至另一合法路線，也就是存在某條 R_l 能夠在不破壞任何限制條件的前提下容納 c_j ，那麼將 c_j 移至 R_l 的某一位置，使得 R_l 在此操作下的增量成本最小化。此策略持續執行，直至整體解完

全合法或沒有進一步的調整空間，然而，如果在不可行的路線中找不到任何可在 Y 不可行的情況下合理重新分配的商店，則該演算法無法實現轉換為可行的目標。在這種情況下，演算法將返回一個不可行的個體 Y ，並透過添加懲罰值來處罰其成本。詳細步驟在圖14。如圖18，路線 R_2 剩餘空間為負值違反重量限制式，其中商店1有可以成功地轉移至另一合法路線 R_1 ，則對此商店進行操作，來優化不可行解。

- (5) 商店不轉移(不共享)階段(Store_Turnto_NoTransfer)：此階段僅有在不可行解優化階段執行後，卻依舊為不可行解時觸發。隨機選取被共享商店，將其轉換為不共享商店，如果沒有共享轉移商店則不執行。如圖19因為上一階段失敗，所以執行此階段，且有共享轉移商店1，所以將此商店轉為不轉移商店，重新加回彈性路線。

透過這五大階段的策略概述，Local Search 旨在對給定問題的解進行系統性和持續的優化。當在處理 CVRP 時，我們只需省略步驟一和步驟五，便可有效進行處理。

Algorithm 5: The pseudocode of Local Search

Input: Y .
1 $Y^* \leftarrow Y$
2 **do**

```

3      Store_Turnto_Transfer( $Y$ )
4       $Y \leftarrow \text{SwappingAlgorithm}(Y)$ 
5      If  $\text{Cost}(Y) < \text{Cost}(Y^*)$ ;
6           $Y^* \leftarrow Y$ 
7           $\text{it\_no\_improvement} \leftarrow 0$ 
8      Else
9           $\text{it\_no\_improvement} \leftarrow \text{it\_no\_improvement} + 1$ 
10     If  $\text{it\_no\_improvement} > 1$ 
11         Break
12      $Y \leftarrow \text{StrongDropOnePoint}(Y)$ 
13     If  $\text{Cost}(Y) < \text{Cost}(Y^*)$ ;
14          $Y^* \leftarrow Y$ 
15          $\text{it\_no\_improvement} \leftarrow 0$ 
16     Else
17          $\text{it\_no\_improvement} \leftarrow \text{it\_no\_improvement} + 1$ 
18     If  $Y$  is infeasible then
19          $Y \leftarrow \text{DropOnePointInfeasible}(Y)$ 
20     If  $Y$  is feasible then
21          $\text{it\_no\_improvement} \leftarrow 0$ 
22     Else
23         Store_Turnto_NoTransfer( $Y$ )
24     While  $\text{it\_no\_improvement} < 2$ ;
25     Return  $Y^*$ 

```

圖 12 The pseudocode of Local Search

Algorithm 6: The pseudocode of Strong Drop One Point

Input: Y

```

1  For  $r \leftarrow 1$  to  $K$  do
2      Forall  $c_q$  on route  $R_r$  of  $Y$  do
3          do
4              remove and feasibly reinsert  $c_q$  on route  $R_b$ ,
                   $R_b \neq R_r$ , in  $Y$  such as its cost is minimum
5              if the cost of  $Y$  reduces;
6  Return  $Y$ 

```

圖 13 The pseudocode of Strong Drop One Point

Algorithm 7: The pseudocode of Drop One Point Infeasible

Input: Y

- 1 **while** exists a store in an infeasible route R_r that can be feasibly reallocated to another route and Y is infeasible **do**
- 2 $c_j \leftarrow$ get a store in a infeasible route R_r that fits on route R_l
- 3 remove c_j from R_r and reinsert it in R_l on position which
The highest enhancement is obtained on R_l
- 4 **Return** Y

圖 14 The pseudocode of Drop One Point Infeasible

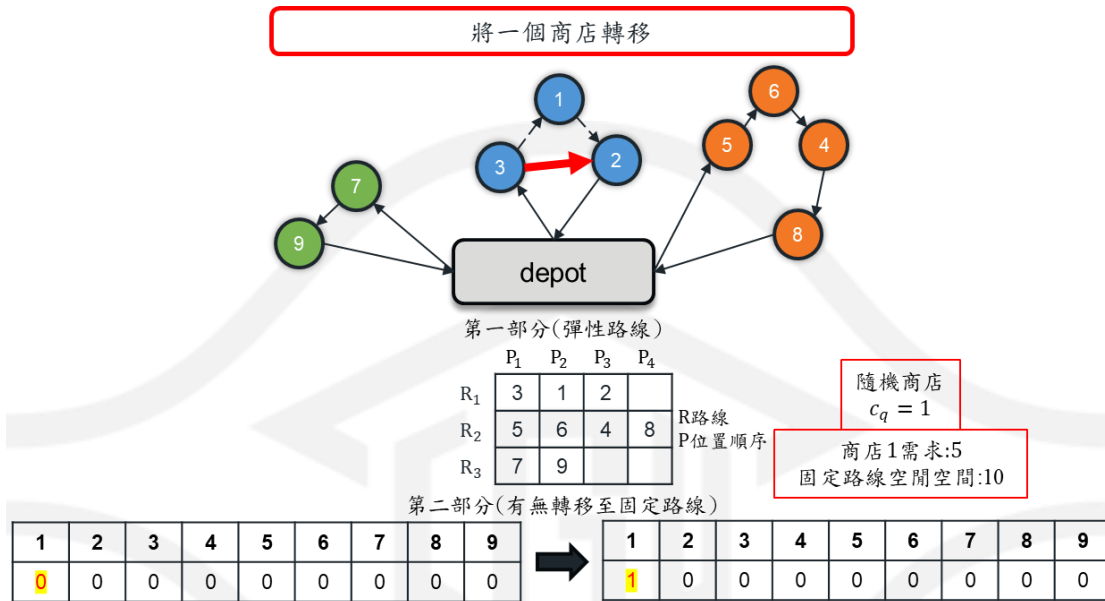


圖 15 The example of Store_Turnto_Transfer

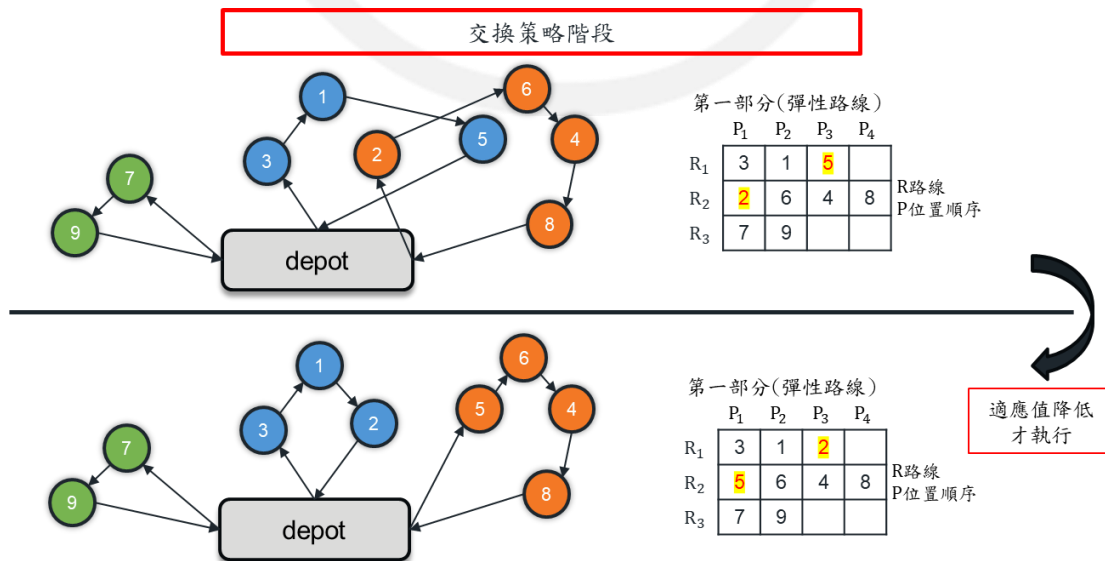


圖 16 The example of Swapping Algorithm

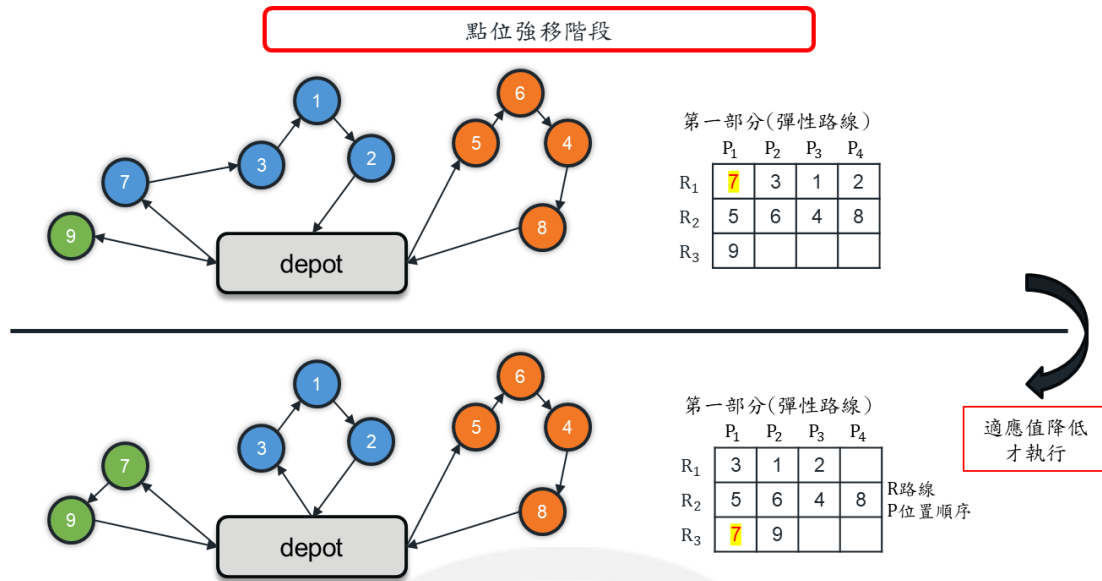


圖 17 The example of Strong Drop One Point

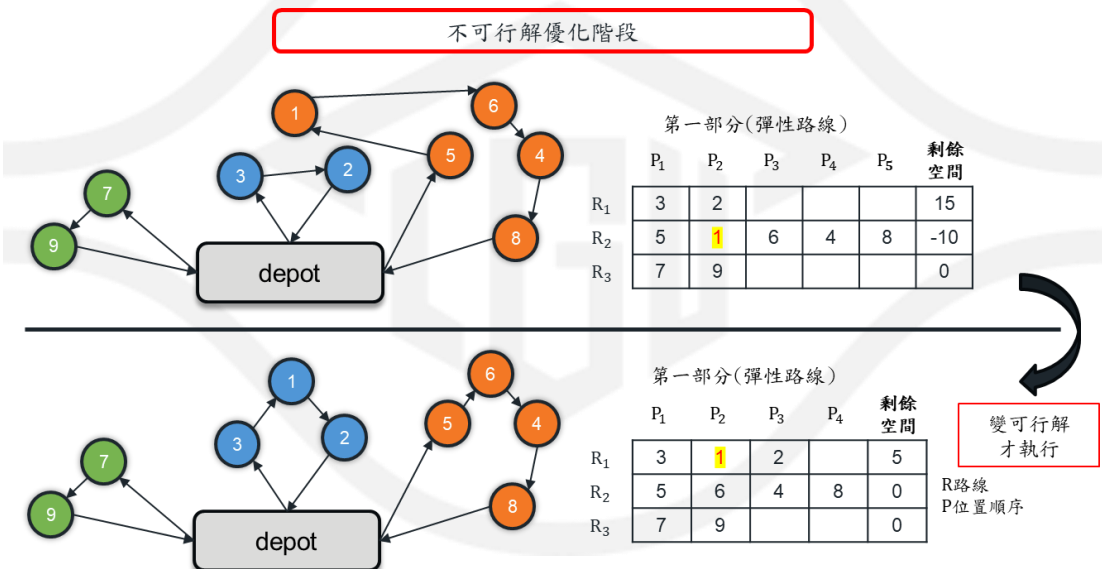


圖 18 The example of Drop One Point Infeasible

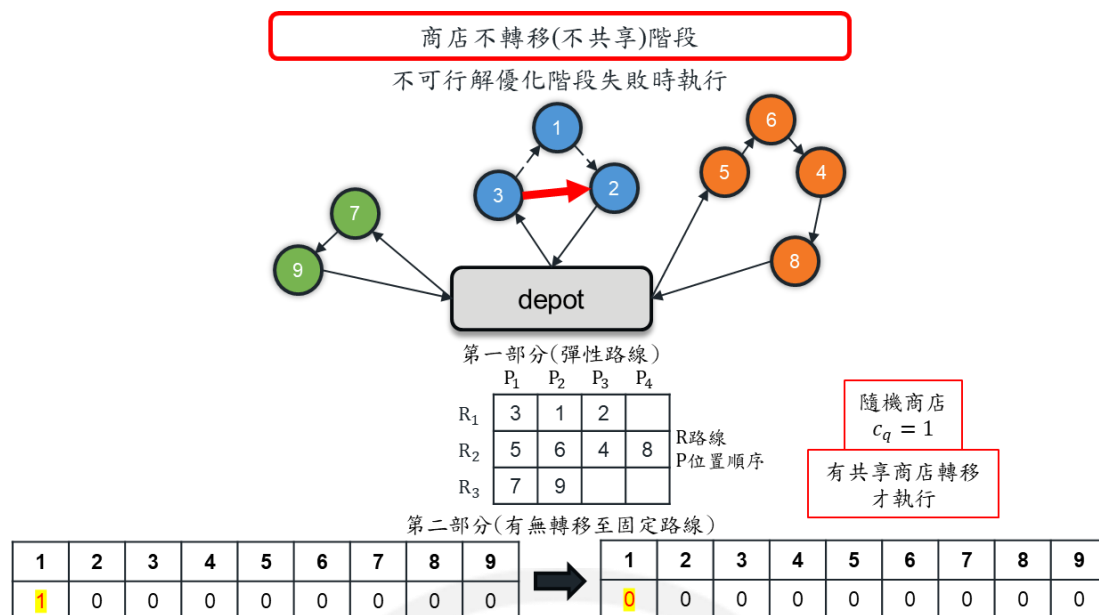


圖 19 The example of Store_Turnto_NoTransfer

3.2.8 選擇操作

在選擇操作(Selection Operator)階段的策略中，完成 Local Search 的個體及其不可行的解都會與其父代解進行詳細比較。此策略的核心機制是，若某個體的成本低於其父代，它將毫無疑問地取代父代。然而，為了增加解空間的多樣性並避免落入局部最優，本研究加入 SA 的機制，即便某些解的成本稍高，演算法也會以某定義的機率接受這些解並允許它們取代父代。

SA 使用 Metropolis 算法生成解序列，類似於將優化問題視為一個多粒子系統。每個解都像是系統中的一個粒子，而目標函數則反映了系統的能量。引入的溫度參數就像是控制系統溫度的旋鈕，而解提供的鄰域和生成機制就是系統中粒子的移動方式。定義了接受原則，

即系統根據接受機制選擇是否接受新的解。簡而言之，模擬退火目標是在全局搜索中找到適應性最佳解，避免陷入局部最小值[70]。Metropolis 算法於公式(11)[58]顯示， $f(i)$ 為上一次能量， $f(j)$ 為當前能量， c 為當前溫度。如果將此公式移除，只剩僅接受較好的解，且更改終止條件，那就是 CDELS，也就是 Souza, et al. [19]中演算法。

$$P_r\{accept\ j\} = \begin{cases} 1, & \text{if } f(j) < f(i) \\ e^{\left(\frac{f(i)-f(j)}{c}\right)}, & \text{else} \end{cases} \quad (11)$$

當遭遇不可行解時，我們會在其原始成本上加上一個特定的逞罰值，以反映其不可行性。然而，這並不意味著這些不可行的個體在演化策略中就被完全忽略了。事實上，這些不可行的個體有時可能包含有價值的結構信息，對後續的世代可能有所助益。例如，即使一個個體有某些部分是不可行的，它仍然可能展現出其他部分的出色結構，這些優秀的結構在演化過程中可能會被其他個體所借鑑學習。

因此，當這些不可行的個體在選擇策略中被認為具有某種價值時，它們會被維持在接下來的世代中，作為演化過程的一部分。然而，最重要的是要認識到，這些不可行的個體，無論它們在策略中的角色如何，都不應被視作最終的優化解，所以在選擇階段中，只有可行解才會更新世代最佳解。選擇階段偽代碼呈現在圖20。

Algorithm 8: The pseudocode of Selection

Input: $X_{i,g}, U'_{i,g+1}, T_k, NP, P_{g+1}$
1 **For** $i = 1$ to NP **do**

```

2      If  $fitness(U'_{i,g+1}) < fitness(X_{i,g})$  or  $\exp(\frac{-\Delta f}{T_k}) > \text{rand}[0,1]$  then
3           $X_{i,g+1} = U'_{i,g+1}$ 
4          If  $(U'_{i,g+1})$  is feasible then
5              If  $fitness(U'_{i,g+1}) < fitness(P_{g+1})$ 
6                   $P_{g+1} = U'_{i,g+1}$ 
7          Else
8               $X_{i,g+1} = X_{i,g}$ 
9      Return  $X_{i,g+1}$ 

```

圖20 選擇操作虛擬碼



第四章 實驗測試與結果分析

本研究進行實驗所使用的電腦設備為 Intel E5-1650CPU@3.50GHz 以及128GB 之記憶體，作業系統環境為 Microsoft Windows 10 64-bit，開發工具為 Visual Studio 2019，使用 C 程式語言開發。

將會在4.1節介紹 CVRP 使用到的題庫，4.2節中說明參數設定，在4.3節進行參數校正實驗找到適當參數，設定完成後會在4.4節測試 CVRP 題庫與說明結果，在4.5節針對 SCRP 問題進行講解，在4.5.1節中說明題庫之生成方式，4.5.2節驗證數學模型之可行性，驗證完再對演算法效益進行測試，最終在4.5.3節進行結果分析。在測試 CVRP 與 SCRP 時會比較 CDELSSA 和 CDELS[19]。

4.1 CVRP 題庫

本研究依照 Souza, et al. [19]所使用之基準題庫，大題庫 A[71]、B[71]、P[71]、E[72]、F[73]還有 M[74]，總共有90個題庫。會利用這些題庫進行參數校正與 CVRP 測試。

4.2 參數設定

本研究依 Paul, et al. [9] 設定將倉庫轉移到實體倉庫用最遠兩點之間的距離一半作為運輸成本，並且 F 突變參數和 CR 交配參數也同樣採用 0.7。CDELSSA 的終止條件為最大降溫次數沒更新。針對 CDELSSA 初始溫度 ST 、降溫速度 CR 、同溫度迭代次數 TI 與最大降溫次數沒更新 TNU ，於 4.3 節使用田口方法在 CVRP 題庫中進行調整。

4.3 參數實驗

考慮四個參數的四個水準，進行 CDELSSA 演算法的校準，表 1 展示各種參數水準與 16 種參數組合。採用了田口 L16 直交實驗設計，在隨機選取的 10 個題庫中各執行 10 次。針對不同組合會使用平均相對百分比偏差 (Average Relative Percentage Deviation, ARPD) 進行計算評估。公式在方程式 (12)，皆用於指定的實例， $C_{max}^{ave}(\Pi)$ 表示在 10 次試驗中使用特定參數組合獲得的平均， $C_{max}^{best}(\Pi)$ 表示在所有參數組合中每種各 10 次試驗中獲得的最佳值。

表 2 呈現了使用不同參數水準所獲得的 ARPD，說明了參數最大降溫次數沒更新 TNU 為最具影響力的因素，因為當 TNU 越大代表演算法迭代次數增加，可以搜索的次數增加。第二個影響關鍵因素是

ST，一開始的溫度將大幅度決定 SA 的效果，第三個為 TNU，在內循環執行次數越多效果越好，同理於迭代次數增加，最後 CR 是影響最小的參數，四個參數最終獲得的值分別為6、0.93、110、100。

$$ARPD = \left(\sum_{r=1}^{10} \frac{C_{max}^{ave}(\Pi) - C_{max}^{best}(\Pi)}{C_{max}^{best}(\Pi)} \times 100\% \right) \div 10 \quad (12)$$

表1 直交數組和獲得的平均相對百分比偏差

No.	ST	CR	TI	TNU	APRD
1	3	0.97	80	40	0.01565
2	3	0.95	90	60	0.00637
3	3	0.93	100	80	0.00567
4	3	0.91	110	100	0.00353
5	6	0.97	90	80	0.00068
6	6	0.95	80	100	0.00321
7	6	0.93	110	40	0.00527
8	6	0.91	100	60	0.00305
9	9	0.97	100	100	0.00136
10	9	0.95	110	80	0.00288
11	9	0.93	80	60	0.00581
12	9	0.91	90	40	0.00825
13	12	0.97	110	60	0.00136
14	12	0.95	100	40	0.00739
15	12	0.93	90	100	0.00116
16	12	0.91	80	80	0.00373

表2 不同水準參數所得到的平均相對百分比偏差

Setting	ST	CR	TI	TNU
1	0.007805	0.004763	0.0071	0.00914
2	0.003053	0.004963	0.004115	0.004148
3	0.004575	0.004478	0.004368	0.00324
4	0.00341	0.00464	0.00326	0.002315
Range	0.004753	0.000485	0.00384	0.006825
Rank	2	4	3	1

4.4 CVRP 測試結果

將 CDELSSA 與其他演算法進行比較如:HGA-VNS[31]、2PH[32]、MGV-CP[33]、DELS[34]、CDELS[19]。比較之結果皆為原始文獻提供，除了 CDELS 是使用提供的原始碼，但此處因為要配合 CDELSSA 終止條件，所以更動其終止條件，導致結果與原始 CDELS 些微落差。每一題皆會各執行10次。最後會將 CDELS 與 CDELSSA 單獨比較，因為 Souza, et al. [19]已表示 CDELS 表現優於其他四種演算法。

表3為 A 題庫結果，其中 $\overline{X}_1$ 為包含所有 A 題庫之平均值， $\overline{X}_2$ 為不包含 A-n45-k7題庫之平均值。CDELSSA 平均結果的 $\overline{X}_1$ 與 $\overline{X}_2$ 皆最接近已知最佳答案(best known solution, BKS)結果。

表3 A 題庫結果

Instance	BKS	DELS	HGA-VNS		MGV-CP		2PH		CDELS		CDELSSA	
		Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best
A-n32-k5	784	784	802	784	784	784	784	784	784	784	784	784
A-n33-k5	661	661	679	661	661	661	661	661	661	661	661	661
A-n33-k6	742	742	760	742	742	742	742	742	742	742	742	742
A-n34-k5	778	778	796	779	778	778	778	778	778	778	778	778
A-n36-k5	799	799	799	799	799	799	799	799	799	799	799	799
A-n37-k5	669	669	669	669	669	669	669	669	669	669	669	669
A-n37-k6	949	949	949	949	949	949	949	949	949	949	949	949
A-n38-k5	730	730	730	730	730	730	730	730	730	730	730	730
A-n39-k5	822	822	822	822	822	822	822	822	822	822	822	822
A-n39-k6	831	831	831	831	831	831	831.2	831	831	831	831	831
A-n44-k6	937	937	937	937	937	937	937	937	937	937	937	937
A-n45-k6	944	944	944	944	945.2	944	945.7	944	944	944	944	944
A-n45-k7	1146	1146	1146	1146	1146	1146	—	—	1146	1146	1146	1146
A-n46-k7	914	914	931	919	914	914	914	914	914	914	914	914
A-n48-k7	1073	1073	1090	1073	1073	1073	1073	1073	1073	1073	1073	1073
A-n53-k7	1010	1010	1010	1010	1010	1010	1010	1010	1010	1010	1010	1010
A-n54-k7	1167	1167	1167	1167	1167	1167	1167	1167	1167	1167	1167	1167

A-n55-k9	1073	1073	1073	1073	1073	1073	1073	1073	1073	1073	1073	1073
A-n60-k9	1354	1354	1354	1354	1354	1354	1354	1354	1354	1354	1354	1354
A-n61-k9	1034	1035	1034	1034	1034	1034	1034	1034	1034	1034	1034	1034
A-n62-k8	1288	1288	1288	1288	1289.4	1288	1288.2	1288	1288	1288	1288	1288
A-n63-k9	1616	1624	1616	1616	1616.2	1616	1616	1616	1616	1616	1616	1616
A-n63-k10	1314	1316	1314	1314	1315.8	1314	1315.5	1314	1314.6	1314	1314.5	1314
A-n64-k9	1401	1416	1401	1401	1402	1401	1401.5	1401	1402.8	1401	1401.9	1401
A-n65-k9	1174	1181	1174	1174	1175.6	1174	1175.5	1174	1174.6	1174	1174	1174
A-n69-k9	1159	1165	1159	1159	1159	1159	1159	1159	1159	1159	1159	1159
A-n80-k10	1763	1779	1763	1763	1763	1763	1764.1	1763	1763.3	1763	1763	1763
$\overline{X_1}$	1041.93	1043.96	1045.85	1042.15	1042.19	1041.93	—	—	1042.05	1041.93	1041.98	1041.93
$\overline{X_2}$	1037.92	1040.04	1042	1038.15	1038.2	1037.92	1038.18	1037.92	1038.05	1037.92	1037.98	1037.92

註:黑粗體表示最佳答案

表4為 B 題庫結果，其中 $\overline{X_1}$ 為包含所有 B 題庫之平均值。

CDELSSA 和 2PH 皆在最佳解部分有找出所有 BKS 結果，但 CDELSSA 平均結果的 $\overline{X_1}$ 較2PH 優秀。CDELS 在平均結果的 $\overline{X_1}$ 擁有最優表現，但相較於 CDELSSA 在最佳解部分 $\overline{X_1}$ 有找出所有 BKS 結果，而 CDELS 沒有。

表4 B 題庫結果

Instance	BKS	DELS	HGA-VNS		MGV-CP		2PH		CDELS		CDELSSA	
		Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best
B-n31-k5	672	672	672	672	672	672	672	672	672	672	672	672
B-n34-k5	788	788	788	788	788	788	788	788	788	788	788	788
B-n35-k5	955	955	955	955	955	955	955	955	955	955	955	955
B-n38-k6	805	805	822	807	805	805	805	805	805	805	805	805
B-n39-k5	549	549	566	552	549	549	549	549	549	549	549	549
B-n41-k6	829	829	829	829	829	829	829	829	829	829	829.9	829
B-n43-k6	742	742	742	742	742	742	742	742	742	742	742	742
B-n44-k7	909	909	921	909	909	909	909	909	909	909	909	909
B-n45-k5	751	751	763	751	751	751	751	751	751	751	751	751
B-n45-k6	678	678	690	678	678	678	678	678	678	678	678	678
B-n50-k7	741	741	753	741	741	741	741	741	741	741	741	741
B-n50-k8	1312	1313	1324	1315	1313	1313	1312.5	1312	1312.2	1312	1312	1312
B-n51-k7	1032	1033	1053	1037	1032	1032	1032	1032	1032	1032	1032	1032
B-n52-k7	747	747	768	751	747	747	747	747	747	747	747	747
B-n56-k7	707	707	728	710	707	707	708.5	707	707	707	707	707
B-n57-k7	1153	1166	1153	1153	1155	1153	1156.5	1153	1153	1153	1153.1	1153
B-n57-k9	1598	1599	1598	1598	1598	1598	1598	1598	1598	1598	1598	1598

B-n63-k10	1496	1504	1496	1496	1496.6	1496	1496	1496	1496.8	1496	1496.8	1496
B-n64-k9	861	861	861	861	861.4	861	861	861	861	861	861	861
B-n66-k9	1316	1322	1316	1316	1316.1	1316	1316.8	1316	1316	1316	1316	1316
B-n67-k10	1032	1032	1032	1032	1033.1	1032	1032	1032	1032.2	1032	1032	1032
B-n68-k9	1272	1281	1272	1272	1272	1272	1273.2	1272	1273.6	1273	1273.3	1272
B-n78-k10	1221	1230	1221	1221	1221	1221	1226.6	1221	1221	1221	1221	1221
$\overline{X_1}$	963.739	965.826	970.565	964.609	963.965	963.783	964.309	963.739	963.861	963.783	963.873	963.739

註:黑粗體表示最佳答案

表5為 P 題庫結果，其中 $\overline{X_1}$ 為包含所有 P 題庫之平均值。CDELSSA 雖然不是唯一在最佳解部分 $\overline{X_1}$ 有找出所有 BKS 結果，但其擁有最優異之平均結果的 $\overline{X_1}$ 。

表5 P 題庫結果

Instance	BKS	DELS	HGA-VNS		MGV-CP		2PH		CDELS		CDELSSA	
		Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best
P-n16-k8	450	450	450	450	450	450	450	450	450	450	450	450
P-n19-k2	212	212	212	212	212	212	212	212	212	212	212	212
P-n20-k2	216	216	216	216	216	216	216	216	216	216	216	216
P-n21-k2	211	211	211	211	211	211	211	211	211	211	211	211
P-n22-k2	216	216	216	216	216	216	216	216	216	216	216	216
P-n22-k8	603	603	603	603	603	603	603	603	603	603	603	603
P-n23-k8	529	533	529	529	529	529	529	529	529	529	529	529
P-n40-k5	458	458	458	458	458	458	458	458	458	458	458	458
P-n45-k5	510	510	510	510	510	510	510	510	510	510	510	510
P-n50-k7	554	554	557	555	554	554	554	554	554	554	554	554
P-n50-k8	631	641	631	631	631.3	631	633.4	631	631.1	631	631	631
P-n50-k10	696	696	706	696	696	696	696	696	696.5	696	696.4	696
P-n51-k10	741	742	759	745	741	741	741	741	741	741	741	741
P-n55-k7	568	568	571	568	568	568	568	568	568	568	568.2	568
P-n55-k10	694	694	694	694	694	694	694	694	694.4	694	695	694
P-n60-k10	744	744	744	744	744	744	744	744	744	744	744	744
P-n60-k15	968	968	979	970	968	968	968	968	968	968	968	968
P-n65-k10	792	792	803	795	792	792	792	792	792	792	792	792
P-n70-k10	827	827	827	827	827	827	827	827	827.1	827	827	827
P-n76-k4	593	593	596	596	594.3	593	593.7	593	594.2	593	593.1	593
P-n76-k5	627	629	627	627	627.5	627	627.3	627	627.6	627	627	627
P-n101-k4	681	685	681	681	681.1	681	681	681	681.1	681	681	681
$\overline{X_1}$	569.136	570.091	571.818	569.727	569.236	569.136	569.291	569.136	569.273	569.136	569.214	569.136

註:黑粗體表示最佳答案

表6為 E、F、M 三個題庫結果，其中 $\overline{X_1}$ 為包含所有題庫之平均值，

僅有 CDELS 和 CDELSSA 擁有， $\overline{X_2}$ 為僅有 E 題庫之平均值。在 $\overline{X_1}$ 結果中 CDELSSA 表現皆優於 CDELS。在 $\overline{X_2}$ 結果中 CDELSSA 雖然不是唯一在最佳解部分 $\overline{X_1}$ 有找出所有 BKS 結果，但其擁有最優異之平均結果的 $\overline{X_1}$ 。

表6 E、F、M 題庫結果

Instance	BKS	DELS		HGA-VNS		MGV-CP		2PH		CDELS		CDELSSA	
		Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.
E-n22-k4	375	375	375	375	375	375	375	375	375	375	375	375	375
E-n23-k3	569	569	569	569	569	569	569	569	569	569	569	569	569
E-n30-k3	534	534	534	534	534	534	534	534	534	534	534	534	534
E-n33-k4	835	835	835	835	835	835	835	835	835	835	835	835	835
E-n51-k5	521	521	521	521	521	521	521	521	521	521	521	521	521
E-n76-k7	682	689	682	682	682	682	682	682	682	682	682	682	682
E-n76-k8	735	738	735	735	735	735	735	735.5	735	735.2	735	735.1	735
E-n76-k10	830	843	830	830	830	830	830.2	830	830.4	830	830.2	830	830
E-n76-k14	1021	1042	1028	1022	1021	1021	1021	1021	1021.6	1021	1021.1	1021	1021
E-n101-k14	1067	1086	1075	1069	1068.3	1067	1068.7	1067	1068.8	1067	1067.6	1067	1067
F-n45-k4	724	-	-	-	-	-	724.4	724	724	724	724	724	724
F-n72-k4	237	-	-	-	-	-	237	237	237	237	237	237	237
F-n135-k7	1162	-	-	-	-	-	1162	1162	1163.9	1163	1162.4	1162	1162
M-n101-k10	820	-	-	-	820	820	820	820	820	820	820	820	820
M-n121-k7	1034	-	-	-	1034	1034	1043.9	1034	1054.6	1034	1034	1034	1034
M-n151-k12	1015	-	-	-	1018	1015	1024	1015	1023.3	1021	1018.5	1015	1015
M-n200-k16	1274	-	-	-	1303.3	1279	-	-	1360.7	1342	1342.6	1331	1331
M-n200-k17	1275	-	-	-	1283.8	1277	-	-	1298.7	1292	1288.5	1283	1283
$\overline{X_1}$	817.22	-	-	-	-	-	-	-	825.233	822.333	822.056	820.833	820.833
$\overline{X_2}$	716.9	723.2	718.4	717.2	717.03	716.9	717.14	716.9	717.2	716.9	717	716.9	716.9

註:黑粗體表示最佳答案，-為原始文獻沒有提供

表7為針對表現最優異兩個演算法 CDELS 與 CDELSSA 輸贏次數比較與平均時間結果，CDELSSA 不管是最佳、平均還是最差適應值的比較次數結果皆呈現輸少贏多的局面。CDELS 與 CDELSSA 在平均時間部分，除了 A 題庫花較多時間外，其餘 CDELSSA 皆使用較

短時間。

表7 CDELS 與 CDELSSA 面對 CVRP 輸贏次數與平均時間結果

題庫	最佳適應值比較			平均適應值比較			最差適應值比較			平均時間	
	贏總計	平手總計	輸總計	贏總計	平手總計	輸總計	贏總計	平手總計	輸總計	CDELS	CDELSSA
A 題庫	0	27	0	4	23	0	3	23	1	90.37	110.33
B 題庫	1	22	0	3	18	2	3	18	2	150.34	117.04
P 題庫	0	22	0	6	14	2	4	17	1	166.38	89.22
E、F、M	4	14	0	9	9	0	6	12	0	2160.04	1852.38
總題庫	5	85	0	22	64	4	16	70	4	538.21	455.30

表8為 CDELS 與 CDELSSA 面對 CVRP 適應值 ARPD t 檢定結果，ARPD 公式在方程式(12)，在最佳、平均與最差適應值 ARPD 的 p 值皆小於0.05，屬於統計上有顯著差異的。

表8 CDELS 與 CDELSSA 面對 CVRP 適應值 ARPD t 檢定

	最佳適應值 ARPD	平均適應值 ARPD	最差適應值 ARPD
差異度	0.000253748	0.000582861	0.001044216
自由度	89	89	89
t 統計	1.866656896	2.050193079	2.124848434
P-Value	6.282078937274E-12	1.107582867612E-18	4.83266127102E-22

4.5 SCRP 測試結果

先於4.5.1節說明 SCRP 題庫生成方法，再於4.5.2節針對小題庫使用求解器驗證演算法效益，最終於4.5.3節分析大題庫結果。每一題皆會各執行10次。

4.5.1 生成 SCRP 題庫

本實驗依 Paul, et al. [9]提出之生成方式，將 CVRP 題庫進行轉變。小型題庫用來驗證數學模型與演算法之正確性，大型題庫用意在於測試演算法效能。

從經典 CVRP 題庫進行改造，基於 Capacitated Vehicle Routing Problem Library 網站所提供的 CVRP 題庫，使用這些題庫所設定的顧客需求與車輛容量，而最佳解作為彈性路線在未共享前的解決方案。小型題庫使用 P-n16-k8、P-n19-k2、P-n23-k8題庫[71]。大型題庫使用 A-n32-k5、A-n48-k7、A-n64-k9和 A-n80-k10題庫[71]，共各四個題庫。生成題庫分為兩步驟：

(1) 固定路線的生成：

先為設定每條路線的最大商店數量(RouteCap)，範圍從2至8，再使用 SA 生成固定路線，一題原生題庫產生共七題。

(2) 固定路線的空間空間生成：

先指定可以轉移的最大商店數量(MaxTransfer)至每條生成的固定路線，MaxTransfer 數量由1至 RouteCap 數量一半商店數，所以一題原生題庫共會產生16題。而每一題中會尋找需求總合最小的商店組合依據 MaxTransfer 限制，最後找到的最小需求總合即為固定路線的

空間空間。

4.5.2 驗證數學模型與演算法效益分析

將生成之小題庫透過 Gurobi 求解器、CDELS 與 CDELSSA 做比較。呈現於表9， $\overline{X_1}$ 為四個題庫 P-n16-k8、P-n19-k2、P-n23-k8、A-n32-k5分別產生題庫之平均值，求解器設定時限為兩小時在執行 A-n32-k5 開始無法求得最佳解，並且除了 CDELS 平均解部分 $\overline{X_1}$ ，其餘皆找到優於求解器解，因此將 A-n32-k5之後題庫設定為大題庫。兩個演算法在處理 P-n16-k8時，還有用 CDELSSA 在面對 P-n19-k2時，最佳解部分 $\overline{X_1}$ 與求解器算得相同結果。在平均解部分 $\overline{X_1}$ CDELSSA 皆較 CDELS 優異。

表9 SCRP 小題庫結果

Route	Max	P-n16-k8					P-n19-k2					P-n23-k8					A-n32-k5				
		Gurobi		CDELS		CDELSSA	Gurobi		CDELS		CDELSSA	Gurobi		CDELS		CDELSSA	Gurobi		CDELS		CDELSSA
Cap	Transfer	Best	Avg.	Best	Avg.	Best	Best	Avg.	Best	Avg.	Best	Best	Avg.	Best	Avg.	Best	Best	Avg.	Best	Avg.	Best
2	1	350	350.6	350	350	350	192	195.1	195	192	192	448	463.1	448	448	448	720	724.8	710	702	700
3	1	361	361.2	361	361	361	194	194	194	194	194	488	491.8	488	488	488	761	762.8	757	741	741
4	1	343	346.6	343	343	343	203	203.2	203	203	203	490	498.1	490	490	490	785	780	773	770	770
	2	258	260.1	258	258	258	179	186.6	183	179	179	362	370.1	362	366.1	365	715	723.7	711	699.4	695
5	1	343	346.6	343	343	343	205	205.7	205	205	205	490	498.1	490	490	490	805	784	784	784	784
	2	258	260.1	258	258.4	258	193	194.5	193	193	193	362	370.1	362	366.1	365	754	764.3	754	743.2	743
6	1	343	346.6	343	343	343	205	209.2	205	205	205	490	498.1	490	490	490	807	784	784	784	784
	2	258	260.1	258	258.4	258	197	197	197	197	197	362	370.1	362	366.1	365	742	744.8	739	737	737
	3	206	206	206	206	206	178	180.8	178	178.2	178	244	247.6	244	244.3	244	726	729.7	721	696.5	694
7	1	343	346.6	343	343	343	205	209.2	205	205	205	490	498.1	490	490	490	805	784	784	784	784
	2	258	260.1	258	258.4	258	195	195.1	195	195	195	362	370.1	362	366.1	365	794	778.8	769	769	769
	3	206	206	206	206	206	180	182.7	180	181.5	180	244	247.6	244	244.3	244	718	743	735	695.6	695
8	1	343	346.6	343	343	343	212	212	212	212	212	490	498.1	490	490	490	750	784	784	784	784
	2	258	260.1	258	258.4	258	194	194	194	194	194	362	370.1	362	366.1	365	732	730.6	725	728	728
	3	206	206	206	206	206	186	187.6	186	186.2	186	244	247.6	244	244.3	244	703	723	714	711.1	710

4	206	206	206	206	206	152	165.9	152	152.7	152	208	209.4	208	208	208	698	703.8	694	690.3	680
$\overline{X_1}$	283.7	285.5	283.7	283.8	283.7	191.8	194.5	192.3	192.0	191.8	383.5	390.5	383.5	384.8	384.4	750.9	752.8	746.1	738.6	737.3

註:黑粗體表示最佳答案

表10為 SCRP 小題庫平均花費時間，求解器在 P-n16-k8、P-n19-k2、P-n23-k8題庫集中皆可以在兩小時內找出最佳解，然而，從 A-n32-k5開始，尋找最優解所需的時間明顯增加，但兩種演算法皆可以用相較較短時間找到解，其中 CDELSSA 的運算時間比 CDELS 還要少。特別是在 p-n19-k2之後的題庫集中，CDELSSA 所花費的時間普遍為最短的。

表10 SCRP 小題庫平均花費秒數

Route	Max	P-n16-k8			P-n19-k2			P-n23-k8			A-n32-k5		
		Gurobi	CDELS	CDELSSA	Gurobi	CDELS	CDELSSA	Gurobi	CDELS	CDELSSA	Gurobi	CDELS	CDELSSA
Cap	Transfer												
2	1	0.15	19.263	5.426	114.922	33.049	5.417	58.455	94.548	17.467	7200.08	376.746	43.79
3	1	0.154	18.691	4.854	21.62	29.235	4.907	147.556	65.916	13.912	7200.066	344.33	31.131
4	1	0.36	18.685	3.726	105.854	26.132	4.842	199.515	70.086	13.333	7200.071	325.738	27.932
	2	0.216	20.063	5.272	46.242	46.119	9.269	189.768	133.563	20.861	7201.352	390.989	45.461
5	1	0.363	19.126	3.587	150.542	24.312	3.712	200.298	64.348	13.094	7200.06	316.002	22.712
	2	0.226	20.733	5.218	255.018	35.755	9.093	95.405	125.587	20.144	7200.056	356.418	38.478
6	1	0.359	20.313	3.583	118.457	22.763	3.456	199.871	61.647	12.318	7200.208	308.466	23.082
	2	0.224	20.838	5.33	184.861	30.816	4.801	175.581	121.721	19.113	7200.072	352.954	34.481
	3	0.106	15.851	2.878	259.018	46.363	9.22	1.221	129.093	15.227	7200.031	420.907	41.049
7	1	0.383	20.301	3.632	117.97	22.059	3.419	200.022	61.679	12.819	7200.049	309.471	17.367
	2	0.226	20.851	5.355	77.222	28.307	4.056	175.339	121.781	19.822	7200.053	338.382	31.914
	3	0.103	16.002	2.863	193.798	35.99	9.049	1.198	128.981	15.657	7200.037	377.98	40.038
8	1	0.359	20.35	3.575	235.375	20.813	3.551	199.731	61.643	13.49	7200.037	300.704	18.32
	2	0.223	20.674	5.343	135.926	28.008	4.147	175.982	121.758	21.111	7200.037	348.447	33.872
	3	0.102	16.014	2.877	250.577	40.353	9.285	1.219	129.07	15.978	7200.045	394.62	39.253
	4	0.106	15.966	2.852	15.938	43.602	9.127	0.197	119.615	7.995	7200.058	432.494	49.098

註:黑粗體表示花費最少時間

4.5.3 分析實驗結果

將生成之大題庫透過 CDELS 與 CDELSSA 做比較。表11為兩個演算法執行生成大題庫 A-n32-k5、A-n48-k7、A-n64-k9和 A-n80-k10 之結果。結果顯示，不論是平均解還是最佳解部分 $\overline{X_1}$ ，CDELSSA 在所有方面都優於 CDELS。表12為時間消耗方面，CDELSSA 同樣在所有測試中超越 CDELS。

表11 SCRP 大題庫結果

Route	Max	A-n32-k5				A-n48-k7				A-n64-k9				A-n80-k10			
		CDELS		CDELSSA		CDELS		CDELSSA		CDELS		CDELSSA		CDELS		CDELSSA	
		Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best
2	1	724.8	710	702	700	968.4	953	876.1	874	1304.3	1268	1180.3	1176	1648.6	1638	1501.7	1489
3	1	762.8	757	741	741	1012.2	988	954.8	953	1331.5	1295	1271.9	1270	1702.6	1677	1603.2	1597
4	1	780	773	770	770	1054.2	1033	1008	1008	1365.1	1347	1308	1308	1735.8	1710	1633.6	1630
	2	723.7	711	699.4	695	963	938	877.4	866	1310.1	1294	1214	1209	1637.3	1622	1517.9	1505
5	1	784	784	784	784	1051.9	1030	1026	1026	1398	1381	1335	1335	1761.6	1756	1671	1671
	2	764.3	754	743.2	743	968.2	949	935.2	932	1314.5	1294	1238.7	1234	1668.1	1653	1565	1552
	1	784	784	784	784	1073	1073	1073	1073	1400	1395	1351	1351	1761.8	1754	1674.1	1674
6	2	744.8	739	737	737	1023	999	973.2	973	1332.8	1322	1273.2	1266	1696.3	1671	1594.1	1583
	3	729.7	721	696.5	694	954.2	937	865.8	855	1273.6	1238	1173.1	1143	1635.8	1620	1493.6	1475
	1	784	784	784	784	1073	1073	1073	1073	1401	1401	1372.6	1372	1763	1763	1727.3	1726
7	2	778.8	769	769	769	1014.5	1007	975	972	1346.6	1329	1294.9	1289	1734.4	1717	1620.8	1614
	3	743	735	695.6	695	952.4	937	912.3	902	1296	1267	1205.5	1196	1651.2	1639	1523.3	1516
	1	784	784	784	784	1073	1073	1073	1073	1401	1401	1364	1364	1763	1763	1765.2	1763
8	2	730.6	725	728	728	1036.1	1013	993.1	993	1357.7	1346	1306.4	1297	1757.2	1745	1642	1642
	3	723	714	711.1	710	959.6	946	921.3	911	1306.8	1298	1239.9	1228	1666.3	1652	1591.3	1579
	4	703.8	694	690.3	680	896.9	860	844.1	829	1263.8	1251	1129.9	1099	1625.5	1607	1486.8	1460
$\overline{X_1}$		752.8	746.1	738.7	737.4	1004.6	988.1	961.3	957.1	1337.7	1320.4	1266.2	1258.6	1700.5	1686.7	1600.7	1592.3

註:黑粗體表示贏過比較算法

表12 SCRP 大題庫平均花費秒數

RouteCap	MaxTransfer	A-n32-k5		A-n48-k7		A-n64-k9		A-n80-k10	
		CDELS	CDELSSA	CDELS	CDELSSA	CDELS	CDELSSA	CDELS	CDELSSA
2	1	376.75	43.79	5452.01	303.43	22928.04	634.47	63516.56	1433.94
3	1	344.33	31.13	4863.04	252.09	17608.33	684.60	62268.24	1533.43
4	1	325.74	27.93	4706.7	206.65	16426.23	510.27	58960.02	1472.06
	2	390.99	45.46	5649.48	306.88	20343.72	708.29	60820.69	1342.19
5	1	316.00	22.71	4232.76	199.67	16519.49	435.29	46831.55	1235.55
	2	356.42	38.48	4321.52	255.89	19363.56	768.35	53737.04	1507.72
6	1	308.47	23.08	3982.34	200.76	17154.45	436.42	53977.51	834.83
	2	352.95	34.48	5025.76	287.40	18044.13	686.95	61525.53	1241.26
	3	420.91	41.05	4923.36	287.40	21397.19	683.66	59879.88	1179.66
7	1	309.47	17.37	3977.94	200.76	15716.05	519.09	47822.99	904.46
	2	338.38	31.91	4388.73	287.02	16835.53	614.20	59097.5	1045.35
	3	377.98	40.04	5237.97	306.99	17711.47	546.38	56043.42	1233.17
8	1	300.70	18.32	3982.32	184.46	16067.57	417.31	47170.03	987.74
	2	348.45	33.87	4632.31	248.68	17683.64	670.51	51284.8	985.37
	3	394.62	39.25	5286.57	278.58	19516.75	639.92	59204.41	1096.34
	4	432.49	49.10	5074.46	313.94	21877.19	681.26	59287.57	1110.89

註:黑粗體表示花費最少時間

表13為 CDELS 與 CDELSSA 面對 SCRP 題庫輸贏次數結果，除了在小題庫最佳適應值比較部分，適應值比較部分 CDELSSA 皆呈現贏多輸少局面。在平均花費時間方面，CDELSSA 可以以更少的時間找到解。

表13 CDELS 與 CDELSSA 面對 SCRP 題庫輸贏次數結果

題庫	最佳適應值比較			平均適應值比較			最差適應值比較			平均時間	
	贏總計	平手總計	輸總計	贏總計	平手總計	輸總計	贏總計	平手總計	輸總計	CDELS	CDELSSA
小題庫	2	41	5	40	8	0	39	9	0	50.59	8.67
大題庫	54	9	1	56	7	1	56	7	1	19969.57	522.50
總題庫	56	50	6	96	15	1	95	16	1	11432.87	302.28

表14為 CDELS 與 CDELSSA 面對 SCRP 題庫適應值 ARPD t 檢定結果，ARPD 公式在方程式(12)，在最佳、平均與最差適應值 ARPD 的 P 值皆小於0.05，屬於統計上有顯著差異的。

表14 CDELS 與 CDELSSA 面對 SCRP 題庫適應值 ARPD t 檢定

	最佳適應值 ARPD	平均適應值 ARPD	最差適應值 ARPD
差異度	0.022584041	0.031953005	0.039698446
自由度	112	112	112
t 統計	7.608113244	10.79096283	12.25392377
P-Value	4.86465E-12	2.67828E-19	1.18E-22

表15為 CDELSSA 平均節省分析，可以看出空間容量越大可以節省之成本與商店也隨之越高，但節省越多商店並不代表會等比例的節省成本，因為轉移也會有一定的轉移花費。

表15 CDELSSA 平均節省分析

RouteCap	MaxTransfer	A-n32-k5		A-n48-k7		A-n64-k9		A-n80-k10	
		平均節省成本	平均節省商店	平均節省成本	平均節省商店	平均節省成本	平均節省商店	平均節省成本	平均節省商店
2	1	10.7%	25.8%	18.5%	44.7%	16.1%	47.6%	15.5%	40.5%
3	1	5.5%	35.5%	11.2%	21.3%	9.4%	31.7%	9.4%	34.2%
4	1	1.8%	25.8%	6.1%	25.5%	6.6%	23.8%	7.5%	25.3%
	2	11.4%	41.9%	19.3%	40.4%	13.7%	38.1%	14.6%	49.4%
5	1	0.0%	0.0%	4.4%	19.1%	4.7%	19.0%	5.2%	20.3%
	2	5.2%	32.3%	13.1%	23.4%	11.9%	36.5%	12.0%	35.4%
6	1	0.0%	0.0%	0.0%	0.0%	3.6%	17.5%	5.0%	17.7%
	2	6.0%	29.0%	9.3%	31.9%	9.6%	25.4%	10.2%	25.3%
	3	11.5%	38.7%	20.3%	44.7%	18.4%	47.6%	16.3%	44.3%
7	1	0.0%	0.0%	0.0%	0.0%	2.1%	15.9%	2.1%	15.2%
	2	1.9%	25.8%	9.4%	23.4%	8.0%	25.4%	8.5%	25.3%
	3	11.4%	35.5%	15.9%	29.8%	14.6%	39.7%	14.0%	40.5%
8	1	0.0%	0.0%	0.0%	0.0%	2.6%	14.3%	0.0%	0.0%
	2	7.1%	22.6%	7.5%	27.7%	7.4%	23.8%	6.9%	25.3%
	3	9.4%	35.5%	15.1%	36.2%	12.3%	39.7%	10.4%	31.6%
	4	13.3%	29.0%	22.7%	36.2%	21.6%	50.8%	17.2%	36.7%
$\bar{x}_1$		5.9%	23.6%	10.5%	25.3%	10.2%	31.1%	9.7%	29.2%

註:皆為節省%數

CDELS 收斂速度慢，會導致需要更長時間才能達到終止條件，從而延長演算法的執行時間，如圖 21 展示了題庫 A-n80-k10 RouteCap8 MaxTransfer4 在 10 次運行中，第三次運行時適應值的變化

情況，該次運行花費的時間相對較長。圖中有上下兩部分，上半部分為前100個執行世代的結果，下半部分則顯示了整體執行世代的結果。當將觀察範圍擴展至全部執行世代時，兩演算法之間的收斂差異變得不那麼明顯，這正是我們特別展示前100個執行世代結果的原因。從這些結果中我們可以明顯觀察到，CDELSSA 在早期階段便能迅速將適應值收斂，相比之下，CDELS 的收斂過程較為緩慢，意味著它需要更多的執行世代來達成相似的收斂效果，所以耗時更長。

因此，本研究新增一個版本引入了一個新的終止條件，加入了時間限制針對大型題庫。我們將這個時間限制設定為 N 乘以20秒，當達到此限制時，算法將停止運行。表16為加入時間終止條件小題庫結果，在 P-n16-k8題庫 CDELSSA 和 CDELS 皆成功找到最佳解。當處理 P-n19-k2題庫時，CDELSSA 能夠找到全部最佳解，而 CDELS 則沒有。至於在 P-n23-k8題庫，CDELS 找到了所有最佳解，但 CDELSSA 則沒有。表17則顯示了加入時間終止條件小題庫平均花費秒數，求解器 Gurobi 在處理 P-n16-k8題庫時可以用極短的時間完成，但在執行 P-n19-k2題庫 CDELSSA 可以花費最少時間，而面對 P-n23-k8題庫共有16題，CDELSSA 可以在12題中取得花費最少時間。

根據表18的結果顯示，不論是平均解還是最佳解部分 $\overline{X_1}$ ，CDELSSA 在所有方面都優於 CDELS。時間消耗方面，如表19所示，

CDELSSA 同樣在所有測試中超越 CDELS。而在表20面對 SCRP 問題的輸贏次數結果中，除了小題庫的最佳適應值比較外，其他最佳、平均或最差的適應值比較，CDELSSA 均展現出勝多於敗的優勢。

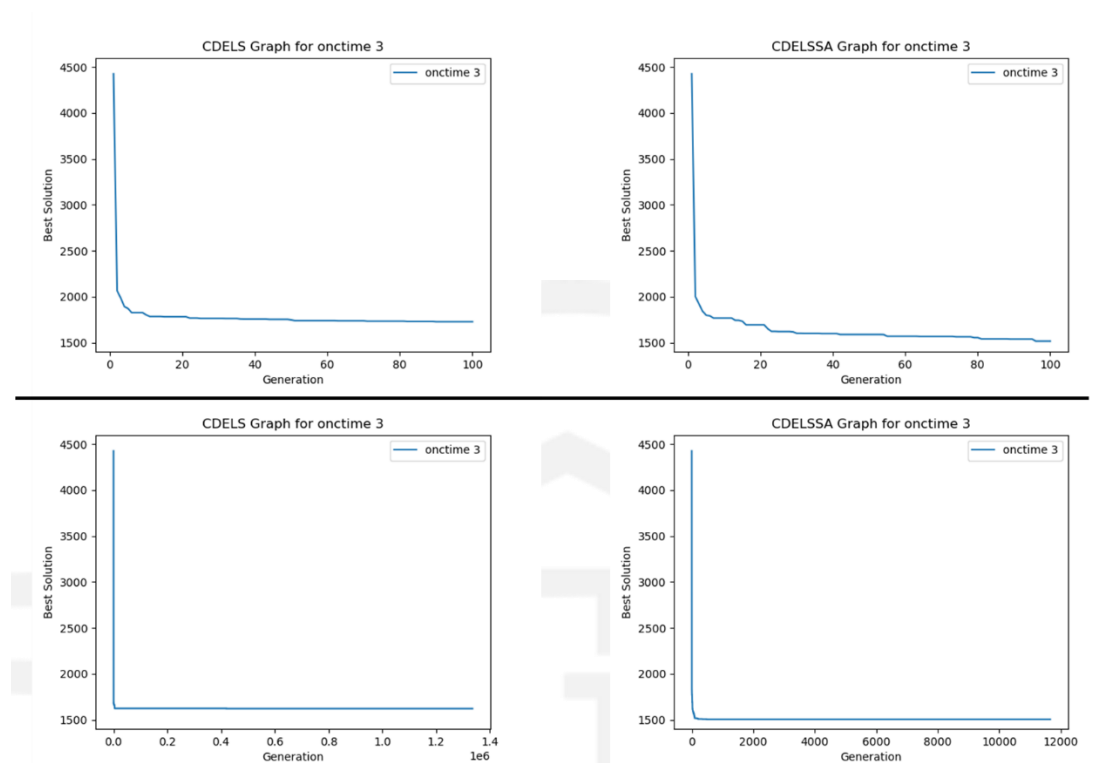


圖21 SCRP A-n80-k10 RouteCap8 MaxTransfer4適應值圖

表16 SCRP 加入時間終止條件小題庫結果

Route	Max	P-n16-k8					P-n19-k2					P-n23-k8				
		Gurobi		CDELS		CDELSSA	Gurobi		CDELS		CDELSSA	Gurobi		CDELS		CDELSSA
		Best	Avg.	Best	Avg.	Best	Best	Avg.	Best	Avg.	Best	Best	Avg.	Best	Avg.	Best
2	1	350	350.6	350	350	350	192	195.1	195	192	192	448	463.1	448	448	448
3	1	361	361.2	361	361.2	361	194	194	194	194	194	488	491.8	488	488.2	488
4	1	343	346.6	343	343	343	203	203.2	203	203	203	490	498.1	490	491.8	490
	2	258	260.1	258	259	258	179	186.6	183	179	179	362	370.1	362	366.4	366
5	1	343	346.6	343	343	343	205	205.7	205	205	205	490	498.1	490	491.8	490
	2	258	260.1	258	259	258	193	194.5	193	193	193	362	370.1	362	366.4	366
	1	343	346.6	343	343	343	205	209.2	205	205	205	490	498.1	490	491.8	490
6	2	258	260.1	258	259	258	197	197	197	197	197	362	370.1	362	366.4	366
	3	206	206	206	206.2	206	178	180.8	178	178.2	178	244	247.6	244	244.1	244
	1	343	346.6	343	343	343	205	209.2	205	205	205	490	498.1	490	491.8	490
7	2	258	260.1	258	259	258	195	195.1	195	195	195	362	370.1	362	366.4	366
	3	206	206	206	206.2	206	180	182.7	180	181.2	180	244	247.6	244	244.1	244

8	1	343	346.6	343	343	343	212	212	212	212	212	490	498.1	490	491.8	490
	2	258	260.1	258	259	258	194	194	194	194	194	362	370.1	362	366.4	366
	3	206	206	206	206.2	206	186	187.6	186	186	186	244	247.6	244	244.1	244
	4	206	206	206	206.2	206	152	165.9	152	153.4	152	208	209.4	208	208	208
$\overline{X_1}$		283.7	285.6	283.7	284.1	283.7	191.8	194.5	192.3	192.1	191.8	383.5	390.5	383.5	385.5	384.8

註:黑粗體表示最佳答案，A-n32-k5為找到優於 Gurobi 解之最優解

表17 SCRP 加入時間終止條件小題庫平均花費秒數

Route	Max	P-n16-k8			P-n19-k2			P-n23-k8		
		Gurobi	CDELS	CDELSSA	Gurobi	CDELS	CDELSSA	Gurobi	CDELS	CDELSSA
2	1	0.15	35.797	9.238	114.922	72.62	11.537	58.455	171.136	26.458
3	1	0.154	35.352	9.155	21.62	65.933	9.563	147.556	101.37	26.767
4	1	0.36	37.223	8.758	105.854	57.695	9.894	199.515	102.761	28.86
	2	0.216	38.202	10.7	46.242	80.624	17.326	189.768	193.084	37.313
5	1	0.363	34.457	8.374	150.542	45.869	7.976	200.298	92.064	27.188
	2	0.226	33.589	10.491	255.018	65.143	17.603	95.405	188.792	34.952
6	1	0.359	32.005	8.367	118.457	42.61	7.193	199.871	91.939	24.568
	2	0.224	33.208	10.533	184.861	52.748	10.085	175.581	177.601	31.541
7	3	0.106	25.527	6.955	259.018	69.362	18.386	1.221	185.938	19.996
	1	0.383	30.604	8.56	117.97	39.263	6.71	200.022	87.015	23.626
	2	0.226	32.023	10.287	77.222	50.902	10.052	175.339	177.364	31.65
8	3	0.103	24.817	7.126	193.798	60.397	18.137	1.198	186.22	19.18
	1	0.359	30.383	8.805	235.375	39.127	6.922	199.731	86.964	23.662
	2	0.223	29.555	10.817	135.926	51.376	8.235	175.982	176.897	31.277
	3	0.102	23.342	6.893	250.577	69.132	17.166	1.219	185.97	19.509
	4	0.106	23.271	6.821	15.938	69.406	16.398	0.197	171.546	13.432

註:黑粗體表示花費較少時間

表18 SCRP 加入時間終止條件大題庫結果

Route	Max	A-n32-k5				A-n48-k7				A-n64-k9				A-n80-k10			
		CDELS		CDELSSA		CDELS		CDELSSA		CDELS		CDELSSA		CDELS		CDELSSA	
		Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.	Best
2	1	724.8	710	701.5	697	969.8	953	874	865	1308.2	1268	1175.5	1171	1656.8	1640	1483.7	1467
3	1	762.8	757	741	741	1013.4	988	954.2	952	1334.2	1295	1269.7	1269	1713	1690	1597.6	1597
4	1	780	773	770	770	1057.2	1033	1008	1008	1369.6	1347	1308	1308	1747.6	1710	1632	1630
	2	723.7	711	700.9	698	963.7	943	873.2	866	1313	1306	1209.8	1198	1642.3	1629	1510.9	1504
5	1	784	784	784	784	1051.9	1030	1026	1026	1399.8	1381	1335	1335	1764.6	1757	1671.1	1671
	2	764.3	754	743.2	743	968.2	949	932.5	930	1319.4	1294	1237.7	1233	1673.8	1653	1558	1544
6	1	784	784	784	784	1073	1073	1073	1073	1404.4	1397	1351	1351	1763.7	1754	1674	1674
	2	744.8	739	737	737	1025.1	999	973	973	1335	1325	1272.3	1264	1711.9	1686	1593.1	1583
	3	729.7	721	695.1	693	954.6	937	864	857	1277.2	1245	1150	1133	1644.6	1630	1486.9	1482
7	1	784	784	784	784	1073	1073	1073	1073	1401.6	1401	1372.2	1372	1765.2	1763	1726	1726
	2	778.8	769	769	769	1014.6	1007	975.4	972	1349.4	1329	1296.7	1289	1741.9	1724	1620	1614
	3	743	735	695.6	695	953.2	937	910.1	899	1297.5	1267	1197.9	1189	1656.3	1645	1519.5	1511

8	1	784	784	784	784	1073	1073	1073	1073	1403.6	1401	1364	1364	1763.6	1763	1764.2	1763
	2	731.7	729	728	728	1037.4	1013	993.3	993	1363.2	1346	1302.7	1291	1762.2	1745	1642	1642
	3	723	714	709.9	702	960.8	951	922.4	917	1308.3	1300	1238.5	1231	1672.7	1657	1583.4	1568
	4	703.8	694	687.4	683	896.9	860	844.1	835	1267	1255	1121.2	1105	1628.6	1612	1494.6	1487
$\overline{X_1}$		752.9	746.38	738.41	737	1005.36	988.69	960.58	957	1340.71	1322.31	1262.64	1256.44	1706.8	1691.13	1597.31	1591.44

註：黑粗體表示贏過比較算法

表19 SCRP 加入時間終止條件大題庫平均花費秒數

RouteCap	MaxTransfer	A-n32-k5		A-n48-k7		A-n64-k9		A-n80-k10	
		CDELS	CDELSSA	CDELS	CDELSSA	CDELS	CDELSSA	CDELS	CDELSSA
2	1	651.043	86.085	980.008	268.435	1300.019	756.948	1620.019	897.312
	3	576.49	61.858	980.009	215.817	1300.016	584.048	1620.038	1072.192
4	1	540.586	55.648	980.011	172.229	1300.018	468.524	1620.034	982.37
	2	634.223	79.782	980.013	266.19	1300.015	704.517	1620.027	1021.603
5	1	532.397	39.045	980.009	161.423	1300.016	414.531	1620.026	745.06
	2	584.463	66.797	980.011	224.496	1300.019	573.833	1620.023	992.252
6	1	516.923	41.677	980.01	167.198	1300.013	354.05	1620.022	587.813
	2	587.259	55.48	980.008	247.38	1300.012	562.756	1620.026	938.169
7	3	643.487	66.852	980.008	264.74	1300.014	680.143	1620.029	1005.912
	1	517.257	31.017	980.007	173.106	1300.011	453.132	1620.025	621.953
8	2	556.654	50.374	980.009	247.681	1300.015	522.931	1620.029	1095.491
	3	607.535	63.306	980.012	243.116	1300.02	488.265	1620.022	1035.667
8	1	508.715	32.226	980.008	156.226	1300.014	345.45	1620.025	1000.045
	2	553.653	52.276	980.008	206.52	1300.013	631.516	1620.033	977.778
8	3	633.57	61.952	980.009	232.205	1300.014	637.93	1620.026	1412.537
	4	660.006	73.707	980.01	271.66	1300.019	606.763	1620.026	1474.689

註：黑粗體表示花費較少時間

表20加入時間終止條件面對 SCRP 題庫總輸贏次數結果

題庫	最佳適應值比較			平均適應值比較			最差適應值比較			平均時間	
	贏總計	平手總計	輸總計	贏總計	平手總計	輸總計	贏總計	平手總計	輸總計	CDELS	CDELSSA
小題庫	2	41	5	39	5	4	33	11	4	79.34	15.73
大題庫	55	9	0	56	7	1	57	7	0	1120.39	454.42
總題庫	57	50	5	95	12	5	90	18	4	674.23	266.41

表21為 CDELS 與 CDELSSA 面對 SCRP 適應值 ARPD t 檢定結果，
 ARPD 公式在方程式(12)，在最佳、平均與最差適應值 ARPD 的 p 值
 皆小於0.05，屬於統計上有顯著差異的。

表21 加入時間終止條件面對 SCRP 適應值 ARPD t 檢定

	最佳適應值 ARPD	平均適應值 ARPD	最差適應值 ARPD
差異度	0.023672112	0.033610769	0.041676806
自由度	111	111	111
t 統計	-7.545170728	-10.4004992	-11.51880594
P-Value	6.69618E-12	2.13196E-18	5.65916E-21

表22為 CDELSSA 平均節省分析，可以看出空間容量越大可以節省之成本與商店也隨之越高，但節省越多商店並不代表會等比例的節省成本，因為轉移也會有一定的轉移花費。

表22 CDELSSA 加入時間終止條件平均節省分析

RouteCap	MaxTransfer	A-n32-k5		A-n48-k7		A-n64-k9		A-n80-k10	
		平均節省成本	平均節省商店	平均節省成本	平均節省商店	平均節省成本	平均節省商店	平均節省成本	平均節省商店
2	1	11.1%	35.5%	19.4%	44.7%	16.4%	49.2%	16.8%	46.8%
	3	5.5%	35.5%	11.3%	23.4%	9.4%	33.3%	9.4%	34.2%
4	1	1.8%	25.8%	6.1%	25.5%	6.6%	23.8%	7.5%	25.3%
	2	11.0%	35.5%	19.3%	40.4%	14.5%	47.6%	14.7%	44.3%
5	1	0.0%	0.0%	4.4%	14.9%	4.7%	19.0%	5.2%	20.3%
	2	5.2%	32.3%	13.3%	25.5%	12.0%	33.3%	12.4%	38.0%
6	1	0.0%	0.0%	0.0%	0.0%	3.6%	17.5%	5.0%	17.7%
	2	6.0%	25.8%	9.3%	31.9%	9.8%	25.4%	10.2%	26.6%
	3	11.6%	38.7%	20.1%	44.7%	19.1%	50.8%	15.9%	35.4%
7	1	0.0%	0.0%	0.0%	0.0%	2.1%	15.9%	2.1%	15.2%
	2	1.9%	25.8%	9.4%	23.4%	8.0%	25.4%	8.5%	26.6%
	3	11.4%	32.3%	16.2%	38.3%	15.1%	39.7%	14.3%	38.0%
8	1	0.0%	0.0%	0.0%	0.0%	2.6%	14.3%	0.0%	0.0%
	2	7.1%	25.8%	7.5%	27.7%	7.9%	25.4%	6.9%	25.3%
	3	10.5%	32.3%	14.5%	31.9%	12.1%	38.1%	11.1%	34.2%
$\bar{X}_1$	4	12.9%	25.8%	22.2%	36.2%	21.1%	49.2%	15.7%	34.2%
		6.0%	23.2%	10.8%	25.5%	10.3%	31.7%	9.7%	28.9%

註:皆為節省%數

第五章 結論與未來展望

本章先於5.1節回顧本研究結果與所做出貢獻，然後於5.2節探討未來研究方向，並提出可以調整與改進方法。

5.1 研究結果與貢獻

本研究提出一種基於 CDELS 並混合了模擬退火法的演算法，稱為組合差分進化與局部搜索混合模擬退火演算法 (Combinatorial Differential Evolution with Local Search hybridize Simulated annealing algorithm, CDELSSA)，該演算法特別針對解決 CVRP 和 SCRP 問題進行優化。為了公平比較，將 CDELS[19]終止條件調整為與 CDELSSA 一致。

在處理 CVRP 時，CDELSSA 在最佳適應值比較中贏5題輸0題，在平均適應值比較中贏22題輸4題，在最差適應值比較中贏16題輸4題，平均花費時間也只使用538.21秒少於 CDELS 的455.30秒。在 SCRP 問題的測試中，CDELSSA 在最佳適應值比較中贏5題輸0題，在平均適應值比較中贏22題輸4題，在最差適應值比較中贏16題輸4題，在 t 檢定中 p-value 值皆小於0.05屬於顯著差異，平均花費時間也只使用538.21秒少於 CDELS 的455.30秒。

面對 SCRP 時，CDELSSA 在最佳適應值比較中贏56題輸6題，在平均適應值比較中贏96題輸1題，在最差適應值比較中贏95題輸1題，平均花費時間也只使用538.21秒少於 CDELS 的455.30秒。在 SCRP 問題的測試中，CDELSSA 在最佳適應值比較中贏5題輸0題，在平均適應值比較中贏22題輸4題，在最差適應值比較中贏16題輸4題，在 t 檢定中 p-value 值皆小於0.05屬於顯著差異，平均花費時間也只使用302.28秒大幅少於 CDELS 的11432.87秒。

為了縮減演算法時間提出另一種加入時間限制終止條件版本，面對 SCRP 時，CDELSSA 在最佳適應值比較中贏57題輸5題，在平均適應值比較中贏95題輸5題，在最差適應值比較中贏90題輸4題，平均花費時間也只使用538.21秒少於 CDELS 的455.30秒。在 SCRP 問題的測試中，CDELSSA 在最佳適應值比較中贏5題輸0題，在平均適應值比較中贏22題輸4題，在最差適應值比較中贏16題輸4題，在 t 檢定中 p-value 值皆小於0.05屬於顯著差異，平均花費時間也只使用266.41秒大幅少於 CDELS 的674.23秒。

CDELSSA 大多贏多輸少展現出超越 CDELS 的表現，另外也針對 CDELS 在執行過程中花費時間較長的問題，而進行終止條件的修整來進行比較，結果並無太大差異，CDELSSA 依舊優於 CDELS 表現。CDELSSA 在節省成本與商店數量上，兩種終止條件之演算法結

果與 Paul, et al. [9]結果相似。

5.2 未來相關探討議題

以下提出幾項建議，為未來的發展方向提供可能的應用途徑：

- (1) 雖然 CDELSSA 可以有效處理 CVRP 與 SCRP，但還是希望可以再進一步提高效率，因此預計進一步增強 CDELSSA 的效能，降低其花費時間，從而能夠更有效地應對具有更多客戶需求的問題。
- (2) 混合元啟發式演算法是學術界的趨勢，CDELSSA 混合兩種演算法也呈現優秀的表現，在未來研究可以將 CDELSSA 再與其他元啟發式演算法進行混合，進行深入的改進和優化。
- (3) 另外將該演算法應用於其他變體的車輛路徑問題，可增強其在真實世界情境下的實用性，考慮引入如時間窗等更多複雜的限制條件。

參考文獻

- [1] Rigby, D., *The future of shopping*. Harvard business review, 2011. **89**(12): p. 65-76.
- [2] Kamel, J. and M. Kay, *Opening the door to omni-channel retailing*. Apparel Magazine, 2011. **53**(2): p. 1-4.
- [3] Verhoef, P.C., P.K. Kannan, and J.J. Inman, *From Multi-Channel Retailing to Omni-Channel Retailing: Introduction to the Special Issue on Multi-Channel Retailing*. Journal of Retailing, 2015. **91**(2): p. 174-181.
- [4] Gallino, S. and A. Moreno, *Integration of online and offline channels in retail: The impact of sharing reliable inventory availability information*. Management Science, 2014. **60**(6): p. 1434-1451.
- [5] Gill, L., *Buzzcommunity: Omni-channel shopping coming to SA*. <http://www.bizcommunity.com/Article/196/182/93333.html>, 2013.
- [6] Verhoef, P.C., P.K. Kannan, and J.J. Inman, *From Multi-Channel Retailing to Omni-Channel Retailing. Introduction to the Special Issue on Multi-Channel Retailing*. Journal of Retailing, 2015. **91**(2): p. 174-181.
- [7] Hübner, A., J. Hense, and C. Dethlefs, *The revival of retail stores via omnichannel operations: A literature review and research framework*. European Journal of Operational Research, 2022. **302**(3): p. 799-818.
- [8] Toth, P. and D. Vigo, *An Overview of Vehicle Routing Problems*, in *The Vehicle Routing Problem*. 2002, Society for Industrial and Applied Mathematics. p. 1-26.

- [9] Paul, J., N. Agatz, R. Spliet, and R.D. Koster, *Shared Capacity Routing Problem – An omni-channel retail study*. European Journal of Operational Research, 2019. **273**(2): p. 731-739.
- [10] Gurobi Optimization , L. *Gurobi optimizer reference manual*. 2023; Available from: <https://www.gurobi.com/>.
- [11] Neslin, S.A., D. Grewal, R. Leghorn, V. Shankar, M.L. Teerling, J.S. Thomas, and P.C. Verhoef, *Challenges and Opportunities in Multichannel Customer Management*. Journal of Service Research, 2006. **9**(2): p. 95-112.
- [12] Cai, Y.-J. and C.K.Y. Lo, *Omni-channel management in the new retailing era: A systematic review and future research agenda*. International Journal of Production Economics, 2020. **229**: p. 107729.
- [13] Sopadjeva, E., U.M. Dholakia, and B. Benjamin, *A study of 46,000 shoppers shows that omnichannel retailing works*. Harvard Business Review, 2017. **3**: p. 1-2.
- [14] Deleersnyder, B., I. Geyskens, K. Gielens, and M.G. Dekimpe, *How cannibalistic is the Internet channel? A study of the newspaper industry in the United Kingdom and The Netherlands*. International Journal of Research in Marketing, 2002. **19**(4): p. 337-348.
- [15] Biyalogorsky, E. and P. Naik, *Clicks and Mortar: The Effect of On-line Activities on Off-line Sales*. Marketing Letters, 2003. **14**(1): p. 21-32.
- [16] Pozzi, A., *The effect of Internet distribution on brick-and-mortar sales*. The RAND Journal of Economics, 2013. **44**(3): p. 569-583.
- [17] Kumar, A., A. Mehra, and S. Kumar, *Why Do Stores Drive Online*

- Sales? Evidence of Underlying Mechanisms from a Multichannel Retailer*. Information Systems Research, 2019. **30**(1): p. 319-338.
- [18] Shockley, J. and G. Fetter, *Distribution co-opetition and multi-level inventory management performance: An industry analysis and simulation*. Journal of Purchasing and Supply Management, 2015. **21**(1): p. 51-63.
- [19] Souza, I.P., M.C.S. Boeres, and R.E.N. Moraes, *A robust algorithm based on Differential Evolution with local search for the Capacitated Vehicle Routing Problem*. Swarm and Evolutionary Computation, 2023. **77**: p. 101245.
- [20] Mor, A. and M.G. Speranza, *Vehicle routing problems over time: a survey*. Annals of Operations Research, 2022. **314**(1): p. 255-275.
- [21] Konstantakopoulos, G.D., S.P. Gayialis, and E.P. Kechagias, *Vehicle routing problem and related algorithms for logistics distribution: a literature review and classification*. Operational Research, 2022. **22**(3): p. 2033-2062.
- [22] Tan, S.-Y. and W.-C. Yeh, *The Vehicle Routing Problem: State-of-the-Art Classification and Review*. Applied Sciences, 2021. **11**(21): p. 10295.
- [23] Van Breedam, A., *Improvement heuristics for the Vehicle Routing Problem based on simulated annealing*. European Journal of Operational Research, 1995. **86**(3): p. 480-490.
- [24] Gendreau, M., A. Hertz, and G. Laporte, *A Tabu Search Heuristic for the Vehicle Routing Problem*. Management Science, 1994. **40**(10): p. 1276-1290.

- [25] Fan, H., Y. Zhang, P. Tian, Y. Lv, and H. Fan, *Time-dependent multi-depot green vehicle routing problem with time windows considering temporal-spatial distance*. Computers & Operations Research, 2021. **129**: p. 105211.
- [26] Storn, R. and K. Price, *Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces*. Journal of Global Optimization, 1997. **11**(4): p. 341.
- [27] Sbai, I., S. Krichen, and O. Limam, *Two meta-heuristics for solving the capacitated vehicle routing problem: the case of the Tunisian Post Office*. Operational Research, 2022: p. 1-43.
- [28] Lin, S.-W., Z.-J. Lee, K.-C. Ying, and C.-Y. Lee, *Applying hybrid meta-heuristics for capacitated vehicle routing problem*. Expert Systems with Applications, 2009. **36**(2): p. 1505-1512.
- [29] Marinakis, Y. and M. Marinaki, *A hybrid genetic-Particle Swarm Optimization Algorithm for the vehicle routing problem*. Expert Systems with Applications, 2010. **37**(2): p. 1446-1455.
- [30] Liang, X., J. Yang, Z. Xiang, and Y. Chen. *Two-stage brain storm optimization-simulated annealing algorithm for constrained vehicle routing problem*. in *2021 3rd International Academic Exchange Conference on Science and Technology Innovation (IAECST)*. 2021. IEEE.
- [31] Sbai, I., S. Krichen, and O. Limam, *Two meta-heuristics for solving the capacitated vehicle routing problem: the case of the Tunisian Post Office*. Operational Research, 2022. **22**(1): p. 507-549.
- [32] Pelletier, S., O. Jabali, and G. Laporte, *The electric vehicle routing*

- problem with energy consumption uncertainty*. Transportation Research Part B: Methodological, 2019. **126**: p. 225-255.
- [33] Machado, A.M., G.R. Mauri, M.C.S. Boeres, and R.d.A. Rosa, *A new hybrid matheuristic of GRASP and VNS based on constructive heuristics, set-covering and set-partitioning formulations applied to the capacitated vehicle routing problem*. Expert Systems with Applications, 2021. **184**: p. 115556.
- [34] Teoh, B.E., S.G. Ponnambalam, and G. Kanagaraj, *Differential evolution algorithm with local search for capacitated vehicle routing problem*. International Journal of Bio-Inspired Computation, 2015. **7**(5): p. 321-342.
- [35] Cattaruzza, D., N. Absi, D. Feillet, and J. González-Feliu, *Vehicle routing problems for city logistics*. EURO Journal on Transportation and Logistics, 2017. **6**(1): p. 51-79.
- [36] Zainudin, S., M. KERWAD, and Z.A. Othman, *A water flow-like algorithm for capacitated vehicle routing problem*. Journal of Theoretical & Applied Information Technology, 2015. **77**(1).
- [37] Prins, C., *A simple and effective evolutionary algorithm for the vehicle routing problem*. Computers & Operations Research, 2004. **31**(12): p. 1985-2002.
- [38] Bäck, T. and H. Schwefel, *An Overview of Evolutionary Algorithms for Parameter Optimization*. Evolutionary Computation, 1993. **1**(1): p. 1-23.
- [39] Holland, J.H., *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and*

artificial intelligence. 1992: MIT press.

- [40] Mühlenbein, H. and G. Paass. *From recombination of genes to the estimation of distributions I. Binary parameters*. in *Parallel Problem Solving from Nature—PPSN IV: International Conference on Evolutionary Computation—The 4th International Conference on Parallel Problem Solving from Nature Berlin, Germany, September 22–26, 1996 Proceedings 4*. 1996. Springer.
- [41] Qiu, X., J.X. Xu, Y. Xu, and K.C. Tan, *A New Differential Evolution Algorithm for Minimax Optimization in Robust Design*. IEEE Transactions on Cybernetics, 2018. **48**(5): p. 1355-1368.
- [42] Zhenzhen, D., Z. Aimin, Z. Guixu, and J. Sanyi. *A differential evolution with an orthogonal local search*. in *2013 IEEE Congress on Evolutionary Computation*. 2013.
- [43] Dai, Z., A. Zhou, G. Zhang, and S. Jiang, *A differential evolution with an orthogonal local search*. 2013. 2329-2336.
- [44] Storn, R. *On the usage of differential evolution for function optimization*. in *Proceedings of North American Fuzzy Information Processing*. 1996.
- [45] Storn, R. and K. Price. *Minimizing the real functions of the ICEC'96 contest by differential evolution*. in *Proceedings of IEEE International Conference on Evolutionary Computation*. 1996.
- [46] Bilal, M. Pant, H. Zaheer, L. Garcia-Hernandez, and A. Abraham, *Differential Evolution: A review of more than two decades of research*. Engineering Applications of Artificial Intelligence, 2020. **90**: p. 103479.

- [47] Das, S. and P.N. Suganthan, *Differential Evolution: A Survey of the State-of-the-Art*. IEEE Transactions on Evolutionary Computation, 2011. **15**(1): p. 4-31.
- [48] Pan, Q.-K., M.F. Tasgetiren, and Y.-C. Liang, *A discrete differential evolution algorithm for the permutation flowshop scheduling problem*. Computers & Industrial Engineering, 2008. **55**(4): p. 795-816.
- [49] Rivera-López, R., E. Mezura-Montes, J. Canul-Reich, and M.A. Cruz-Chávez, *A permutational-based Differential Evolution algorithm for feature subset selection*. Pattern Recognition Letters, 2020. **133**: p. 86-93.
- [50] Liu, H., Z. Cai, and Y. Wang, *Hybridizing particle swarm optimization with differential evolution for constrained numerical and engineering optimization*. Applied Soft Computing, 2010. **10**(2): p. 629-640.
- [51] Stanarevic, N., *Hybridizing artificial bee colony (ABC) algorithm with differential evolution for large scale optimization problems*. International Journal of Mathematics and Computers in Simulation, 2012. **6**(1): p. 194-202.
- [52] Sharma, T.K. and M. Pant, *Differential Operators Embedded Artificial Bee Colony Algorithm*, in *Modeling Applications and Theoretical Innovations in Interdisciplinary Evolutionary Computation*, W.-C. Samuelson Hong, Editor. 2013, IGI Global: Hershey, PA, USA. p. 149-163.
- [53] Ustun, D. and A. Akdagli. *A study on the performance of the hybrid optimization method based on artificial bee colony and differential*

- evolution algorithms*. in *2017 International Artificial Intelligence and Data Processing Symposium (IDAP)*. 2017.
- [54] Worasuchee, C. *An opposition-based hybrid Artificial Bee Colony with differential evolution*. in *2015 IEEE Congress on Evolutionary Computation (CEC)*. 2015.
- [55] Sethanan, K. and R. Pitakaso, *Improved differential evolution algorithms for solving generalized assignment problem*. Expert Systems with Applications, 2016. **45**: p. 450-459.
- [56] Li, H., Y.-C. Jiao, and L. Zhang, *Hybrid differential evolution with a simplified quadratic approximation for constrained optimization problems*. Engineering Optimization, 2011. **43**(2): p. 115-134.
- [57] Metropolis, N., A.W. Rosenbluth, M.N. Rosenbluth, A.H. Teller, and E. Teller, *Equation of state calculations by fast computing machines*. The journal of chemical physics, 1953. **21**(6): p. 1087-1092.
- [58] Delahaye, D., S. Chaimatanan, and M. Mongeau, *Simulated Annealing: From Basics to Applications*, in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Editors. 2019, Springer International Publishing: Cham. p. 1-35.
- [59] Kirkpatrick, S., C.D. Gelatt Jr, and M.P. Vecchi, *Optimization by simulated annealing*. science, 1983. **220**(4598): p. 671-680.
- [60] Osman, I.H., *Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem*. Annals of operations research, 1993. **41**: p. 421-451.
- [61] Tavakkoli-Moghaddam, R., N. Safaei, M.M.O. Kah, and M. Rabbani, *A new capacitated vehicle routing problem with split service for*

- minimizing fleet cost by simulated annealing*. Journal of the Franklin Institute, 2007. **344**(5): p. 406-425.
- [62] Xiao, Y., Q. Zhao, I. Kaku, and N. Mladenovic, *Variable neighbourhood simulated annealing algorithm for capacitated vehicle routing problems*. Engineering Optimization, 2014. **46**(4): p. 562-579.
- [63] Hou, Y., B. Liu, L. Dang, W. He, and W. Gu, *A Local Search-based Metaheuristic Algorithm Framework for the School Bus Routing Problem*. Engineering Letters, 2022. **30**(1): p. 17-28.
- [64] Kaboudani, Y., S.H. Ghodsypour, H. Kia, and A. Shahmardan, *Vehicle routing and scheduling in cross docks with forward and reverse logistics*. Operational Research, 2020. **20**(3): p. 1589-1622.
- [65] Rajabi-Bahaabadi, M., A. Shariat-Mohaymany, M. Babaei, and D. Vigo, *Reliable vehicle routing problem in stochastic networks with correlated travel times*. Operational Research, 2021. **21**(1): p. 299-330.
- [66] Mar'i, F., W.F. Mahmudy, and P.B. Santoso. *Hybrid Particle Swarm Optimization and Simulated Annealing for Capacitated Vehicle Routing Problem*. in *2019 International Conference on Sustainable Information Engineering and Technology (SIET)*. 2019.
- [67] Ilhan, İ., *A population based simulated annealing algorithm for capacitated vehicle routing problem*. Turkish Journal of Electrical Engineering and Computer Sciences, 2020. **28**(3): p. 1217-1235.
- [68] Askarzadeh, A., L.d.S. Coelho, C.E. Klein, and V.C. Mariani. *A population-based simulated annealing algorithm for global*

- optimization. in *2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2016.
- [69] İlhan, İ., *An improved simulated annealing algorithm with crossover operator for capacitated vehicle routing problem*. Swarm and Evolutionary Computation, 2021. **64**: p. 100911.
- [70] Kirkpatrick, S., C.D. Gelatt, and M.P. Vecchi, *Optimization by Simulated Annealing*. Science, 1983. **220**(4598): p. 671-680.
- [71] Augerat, P., D. Naddef, J.M. Belenguer, E. Benavent, A. Corberan, and G. Rinaldi, *Computational results with a branch and cut code for the capacitated vehicle routing problem*. 1995: France.
- [72] Christofides, N. and S. Eilon, *An algorithm for the vehicle-dispatching problem*. Journal of the Operational Research Society, 1969. **20**: p. 309-318.
- [73] Fisher, M.L., *Optimal solution of vehicle routing problems using minimum k-trees*. Operations research, 1994. **42**(4): p. 626-642.
- [74] Christofides, N., A. Mingozzi, and P. Toth, *Loading problems*. N. Christofides and al., editors, Combinatorial Optimization, 1979: p. 339-369.